

Computer Science

Mr Nouredine Tadjerout

Computer Science

Paper 1- Chapter 3: Hardware

3.1 Computer architecture

- 1- (a): Understand the role of the central processing unit (CPU) in a computer.
(b): Understand what is meant by a microprocessor.
- 2- (a): Understand the purpose of the components in a CPU, in a computer that has a Von Neumann architecture.

(b): Describe the process of the fetch–decode–execute cycle including the role of each component in the process.

Computer Science

Paper 2 - Chapter 7: Algorithm design and problem-solving

7.4 Algorithm design and problem-solving

4 - Understand standard methods of solution.

- linear search
- bubble sort

Resources



Craig 'n' Dave For Students

CRAIG 'N' DAVE

To explore the entire series,
visit <https://craigndave.org/free-videos/>

Resources



To explore the entire series,
visit www.hoddereducation.com/cambridge-igcse-computerscience

Computer Science

Paper 1- Chapter 3: Hardware

3.1 Computer architecture

- 1- (a): Understand the role of the central processing unit (CPU) in a computer.
(b): Understand what is meant by a microprocessor.
- 2- (a): Understand the purpose of the components in a CPU, in a computer that has a Von Neumann architecture.

(b): Describe the process of the fetch–decode–execute cycle including the role of each component in the process.

Learning Objectives

3.1 Computer architecture

Candidates should be able to:

- 1 (a) Understand the role of the central processing unit (CPU) in a computer
- (b) Understand what is meant by a microprocessor
- 2 (a) Understand the purpose of the components in a CPU, in a computer that has a Von Neumann architecture
- (b) Describe the process of the fetch–decode–execute cycle including the role of each component in the process

Notes and guidance

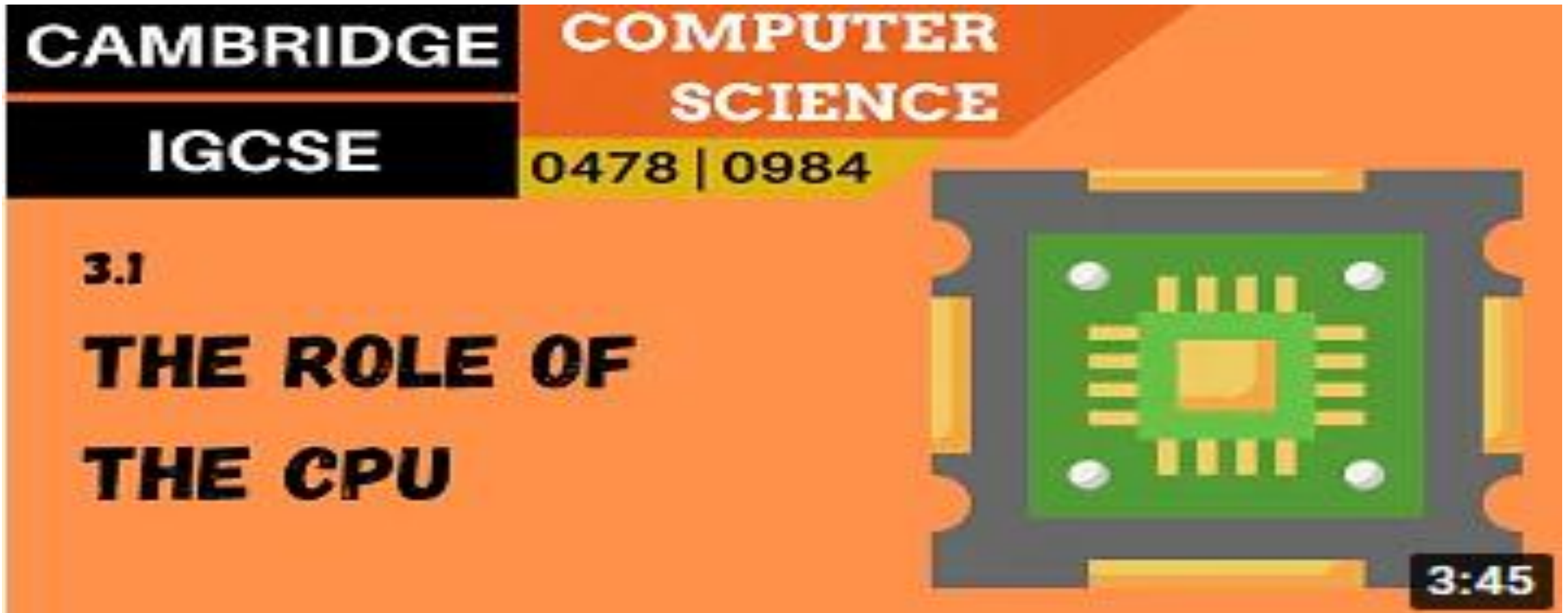
- The CPU processes instructions and data that are input into the computer so that the result can be output
- A microprocessor is a type of integrated circuit on a single chip
- Including:
 - units: arithmetic logic unit (ALU) and control unit (CU)
 - registers: program counter (PC), memory address register (MAR), memory data register (MDR), current instruction register (CIR) and accumulator (ACC)
 - buses: address bus, data bus and control bus
- How instructions and data are fetched from random access memory (RAM) into the CPU, how they are processed using each component and how they are then executed
- Storing data and addresses into specific registers
- Using buses to transmit data, addresses and signals
- Using units to fetch, decode and execute data and instructions

3.1:Computer Architecture

Recap Pervious section 3.1.1 (a & b) and 3.1.2 (a)

Watch the video for The role of the central processing unit

<https://www.youtube.com/watch?v=clL1AuQNHmQ>



3.1:Computer Architecture

The role of the central processing unit



The role of the CPU

The **central processing unit (CPU)**:

- takes in data and instructions;
- processes them; and
- produces output.

The terms **CPU** and **microprocessor** essentially mean the same thing.

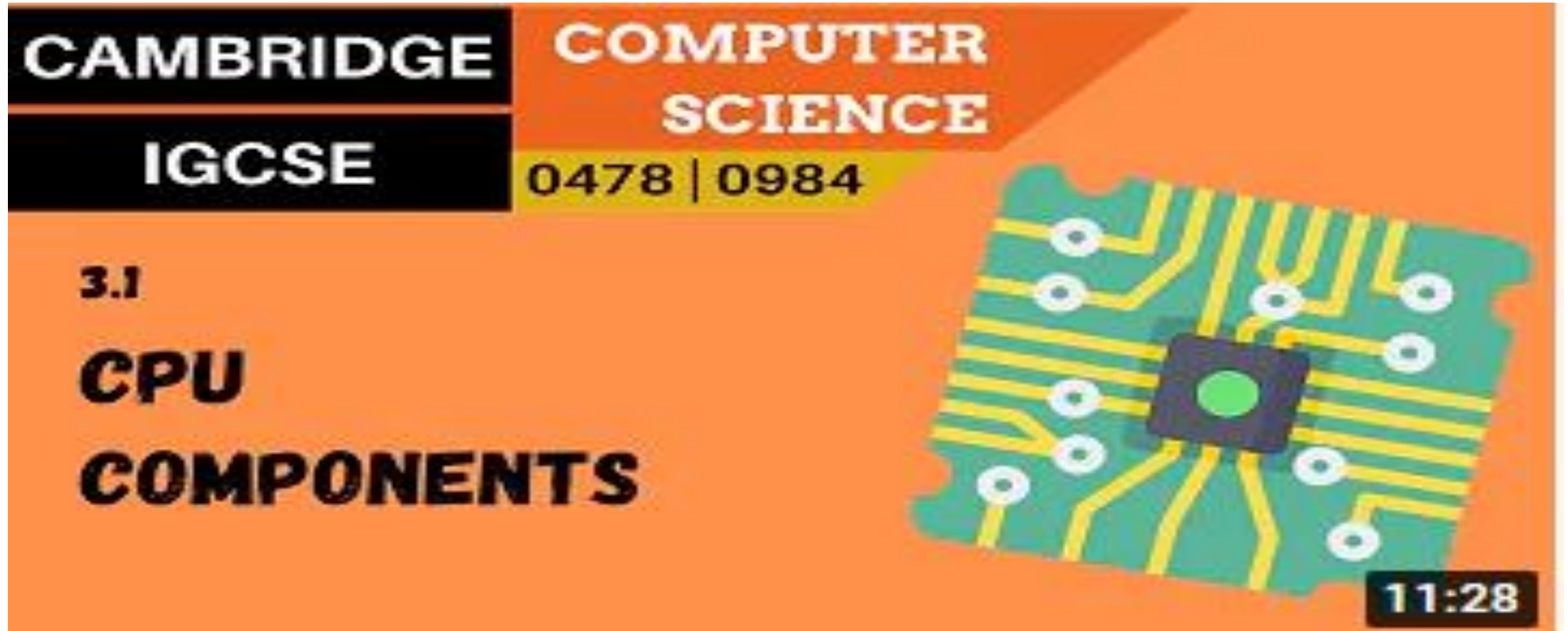
A **CPU/microprocessor** is typically installed as an integrated circuit on a single microchip.

3.1:Computer Architecture

Recap Pervious section 3.1.1 (a & b) and 3.1.2 (a)

Watch the video for Purpose of the components in a CPU

<https://www.youtube.com/watch?v=K9EuHc5kzJc>



3.1:Computer Architecture

Components in a CPU



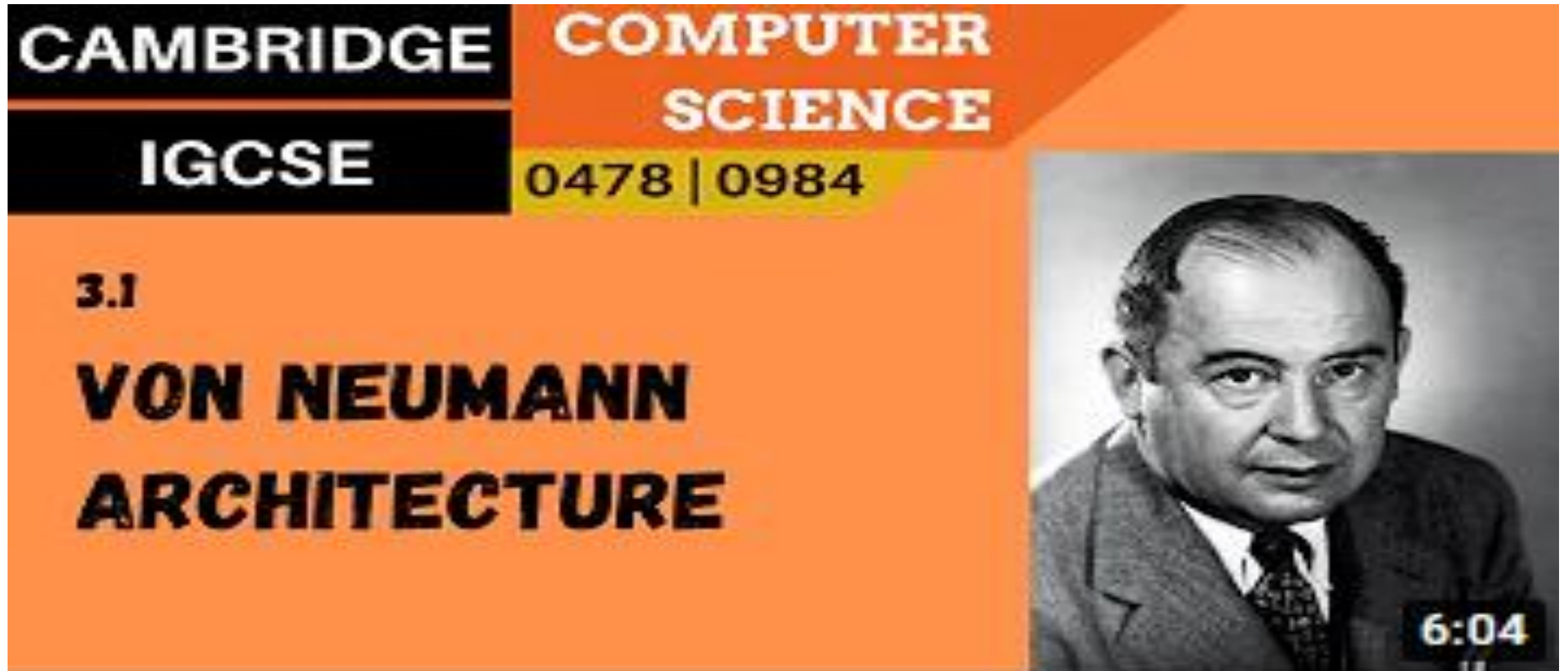
Components of the CPU

The CPU consists of the following components:

- **Arithmetic logic unit (ALU):** Performs calculations and logical decisions.
- **Control unit (CU):** Sends signals to control how data moves around the CPU.
- **Buses:** A collection of wires through which data and instructions are transmitted from one component to another.
 - **Address bus:** Unidirectional – carries the addresses that data needs to be written to or read from.
 - **Data bus:** Bidirectional – carries actual data or instructions.
 - **Control bus:** Bidirectional – carries command and control signals telling components when they should be receiving reads, writes, etc.
- **Registers:** Tiny, super-fast pieces of onboard memory inside the CPU, each with a very specific purpose.
 - **Program counter (PC):** Holds the address of the next instruction to be executed.
 - **Memory address register (MAR):** Holds the address of the memory location that data or an instruction is to be fetched from or written to.
 - **Memory data register (MDR):** Used to temporarily store data that is read from or written to memory.
 - **Current instruction register (CIR):** Holds the current instruction being executed.
 - **Accumulator (ACC):** One of several general-purpose registers. The results of calculations carried out by the ALU can be temporarily stored here.

3.1:Computer Architecture

<https://www.youtube.com/watch?v=C0Pch0zpsT4>



3.1:Computer Architecture



Von Neumann architecture

The von Neumann architecture consists of a:

- Control unit (CU)
- Arithmetic logic unit (ALU)
- Main memory (RAM)
- Inputs and outputs

The von Neumann architecture is based on the stored program concept.

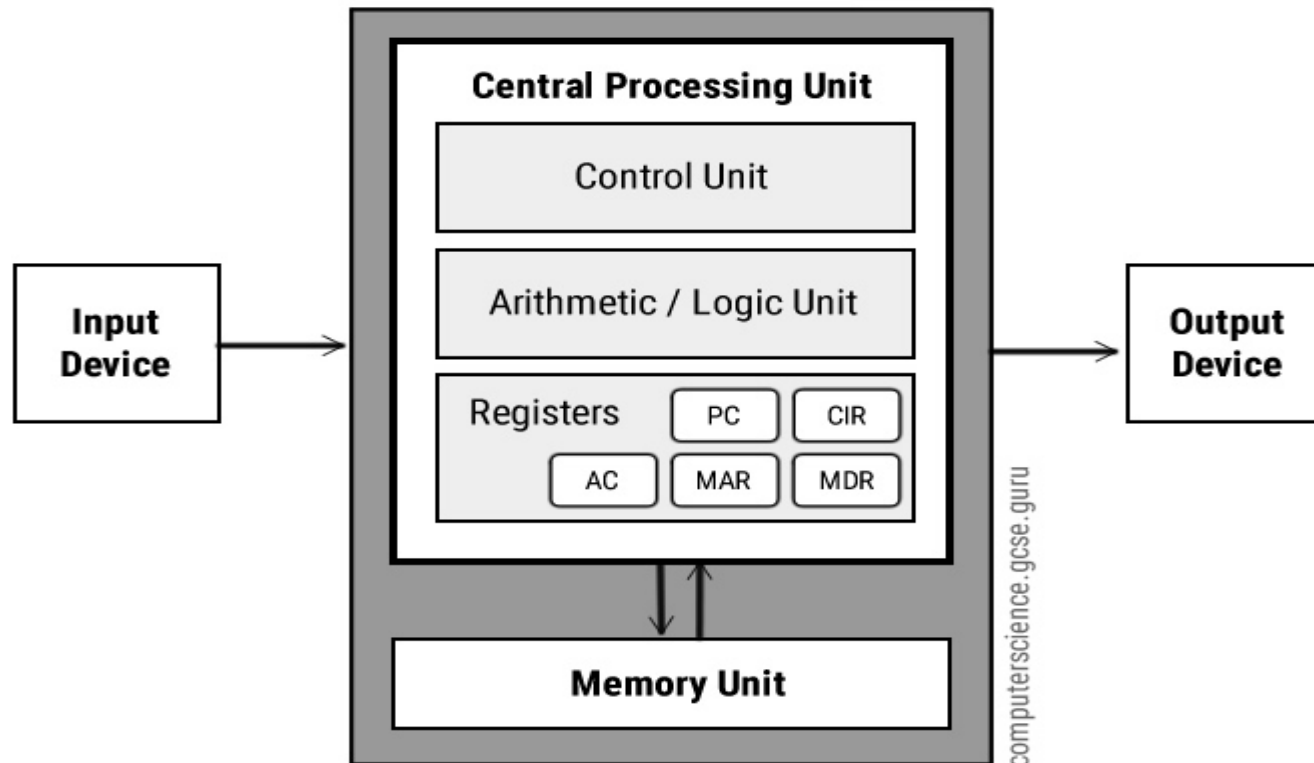
Both instruction data *and* program data are stored in the same memory in binary form. There is no way to know if the pure binary held in memory represents instructions or data simply by looking at it.

A von Neumann CPU contains the following registers:

- Program counter (PC): Holds the address of the next instruction to be executed in memory.
- Memory address register (MAR): Holds the memory address where data is to be fetched from or written to.
- Memory data register (MDR): Holds data fetched from or to be written to memory.
- Accumulator: Holds the results of calculations.

3.1:Computer Architecture

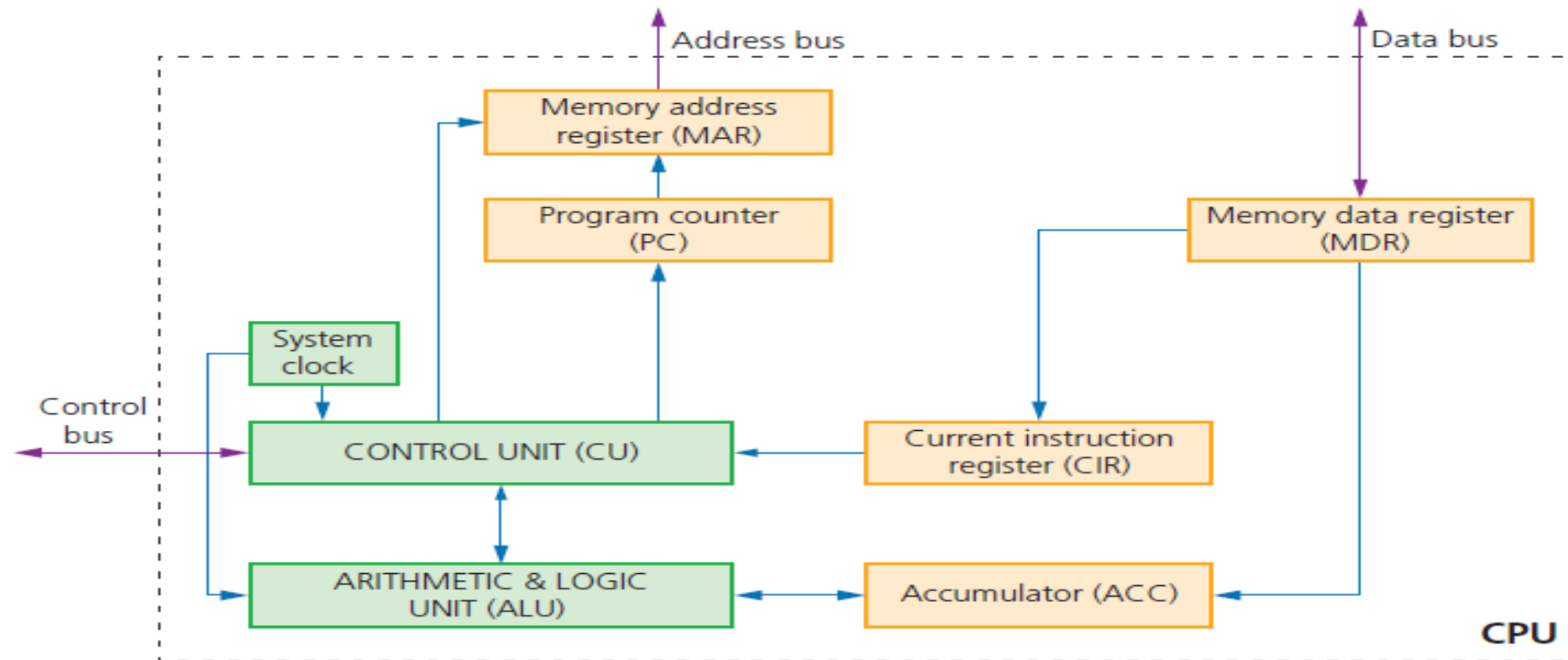
Von Neumann computer architecture design consists of a [Control Unit](#), [Arithmetic and Logic Unit \(ALU\)](#), [Memory Unit](#) and [Registers](#) and [Inputs/Outputs](#). Von Neumann architecture is based on the stored-program computer concept, where instruction data and program data are stored in the same memory. This design is still used in most computers produced today.



3.1:Computer Architecture

Von Neumann layout

Figure below shows a slightly more detailed diagram of the von Neumann architecture. It brings to our attention another new concept in this computer model – the idea of **ADDRESSES** and **REGISTERS**. Addresses indicate where the data is stored and registers are needed so that data can be manipulated within the computer..



3.1:Computer Architecture

Control Unit

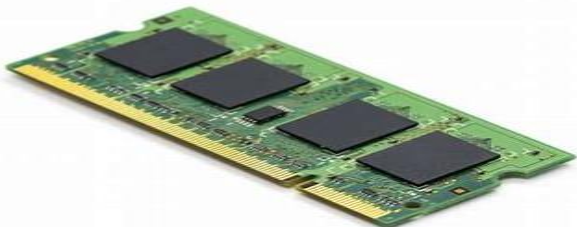
The control unit controls the operation of the CPU, memory and input/ output devices.

The job of the control unit is to coordinate and control all the other parts of the CPU. It tells the registers & ALU what to do with an instruction that is passed to them.

The CU decodes the instructions retrieved from memory and instructs the ALU to perform an arithmetic or logic instruction on the data from the decoded instruction.

Arithmetic and logic unit (ALU) (or Arithmetic Logic Unit): The ALU allows arithmetic (add, subtract etc) and logic (AND, OR, NOT etc) operations to be carried out. ALU to do calculations and comparisons and ALU is responsible for the arithmetic or logic processing requirements of the instructions in a running program.

RAM (random access memory) is a computer's short-term memory, where the data that the processor is currently using is stored. Your computer can access RAM memory much faster than data on a hard disk, SSD, or other long-term storage device, which is why RAM capacity is critical for system performance.



3.1:Computer Architecture

Registers

One of the most fundamental components of the von Neumann system are the **registers**. Registers can be general or special purpose. We will only consider the special purpose registers. The use of these registers is explained more fully in the Fetch–Decode–Execute cycle

▼ **Table 3.1** Specific purpose registers

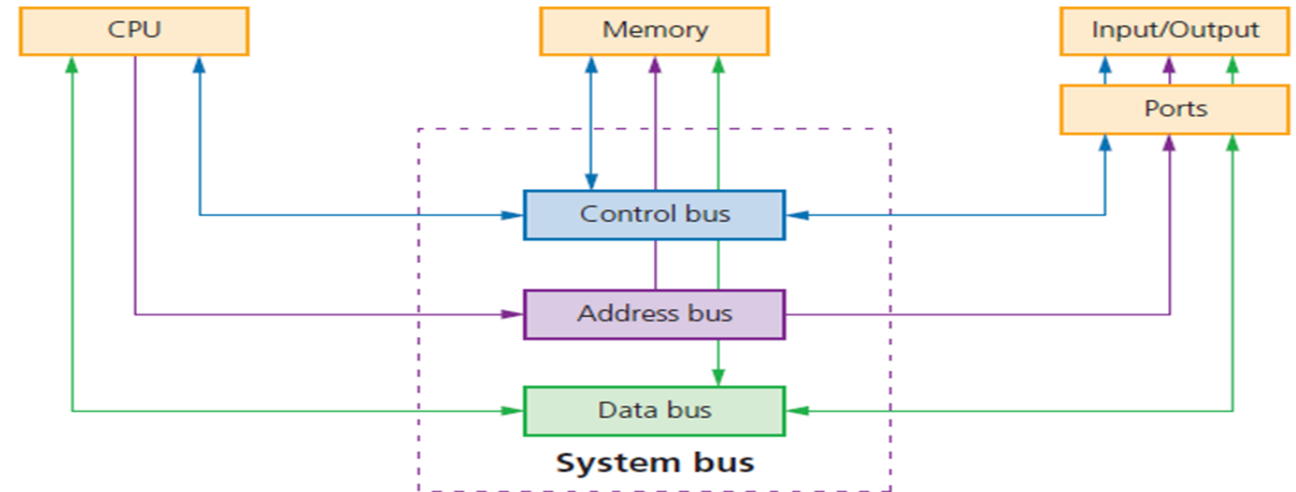
Register	Abbreviation used	Function/purpose of register
current instruction register	CIR	this register stores the current instruction being decoded and executed
accumulator	ACC	this register is used when carrying out ALU calculations; it stores data temporarily during the calculations
memory address register	MAR	this register stores the address of the memory location currently being read from or written to
memory data/ buffer register	MDR	this register stores data which has just been read from memory or data which is about to be written to memory
program counter	PC	this register stores the address where the next instruction to be read can be found

3.1:Computer architecture

System buses

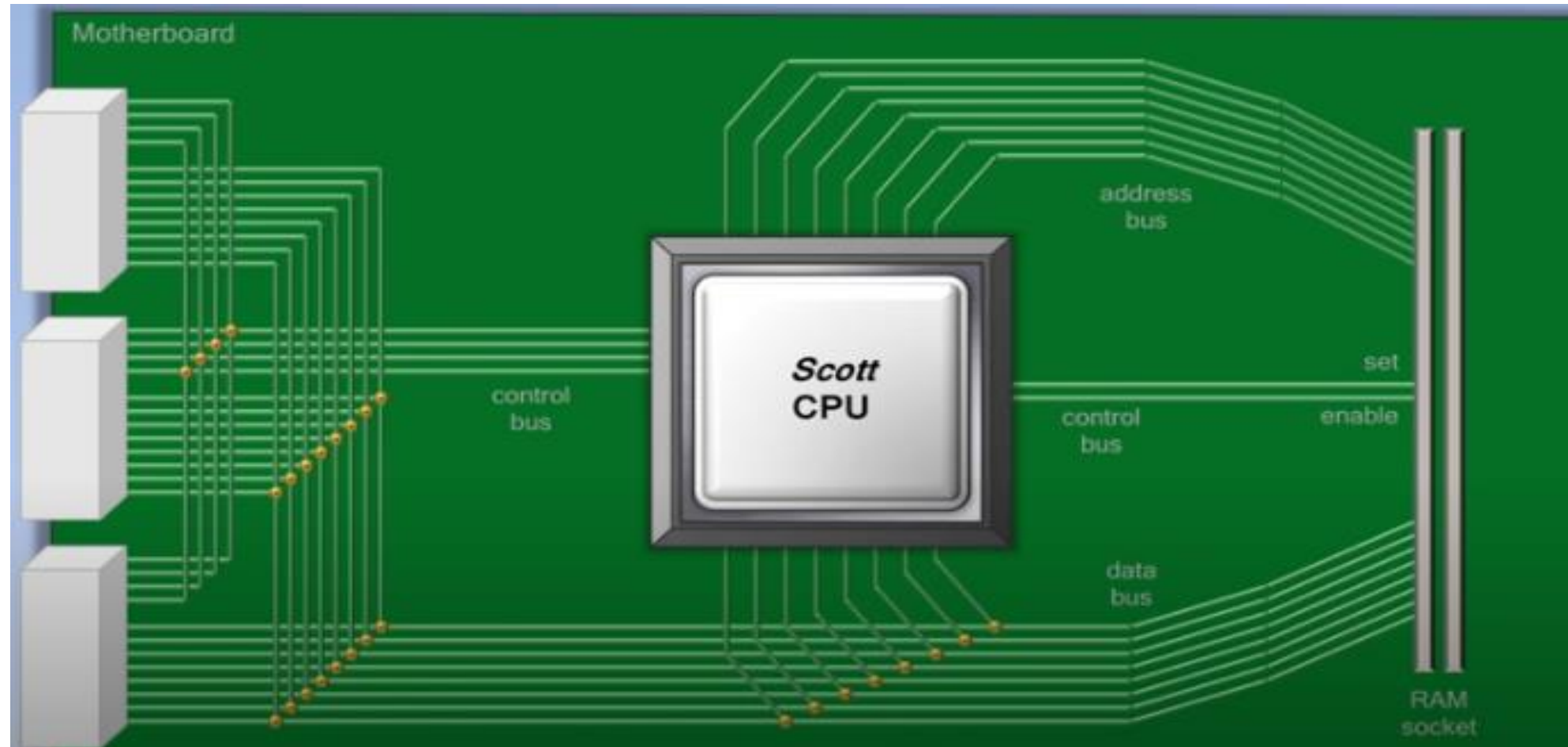
Busess essentially move data around the computer and also send out control signals to make sure everything is properly synchronized. Also Figure below shows how these buses are used to connect the CPU to the memory and to input/output devices.

Type of bus	Description of bus	Data/signal direction
address bus	carries signals relating to addresses (see later) between the processor and the memory	unidirectional (signals travel in one direction only)
data bus	sends data between the processor, the memory unit and the input/output devices	bi-directional (data can travel in both directions)
control bus	carries signals relating to the control and coordination of all activities within the computer (examples include: the read and write functions)	unidirectional (signals travel in one direction only)



Address Bus	Carries the addresses of data (but not the data) between the processor and memory
Data Bus	Carries data between the processor, the memory unit and the input/output devices
Control Bus	Carries control signals/commands from the CPU (and status signals from other devices) in order to control and coordinate all the activities within the computer

System buses



Independent or Pair Work

IGCSE Exam Question 1

Name **three** different buses that are used in the fetch-execute cycle.

Bus 1

Bus 2

Bus 3

[3]

IGCSE Exam Answer 1

Question	Answer	Marks
1	☐ address (bus) ☐ control (bus) ☐ data (bus)	3

IGCSE Exam Question 2

(a) One of the key features of von Neumann computer architecture is the use of buses.

Three buses and three descriptions are shown below.

Draw a line to connect each bus to its correct description.

Bus

address bus

control bus

data bus

Description

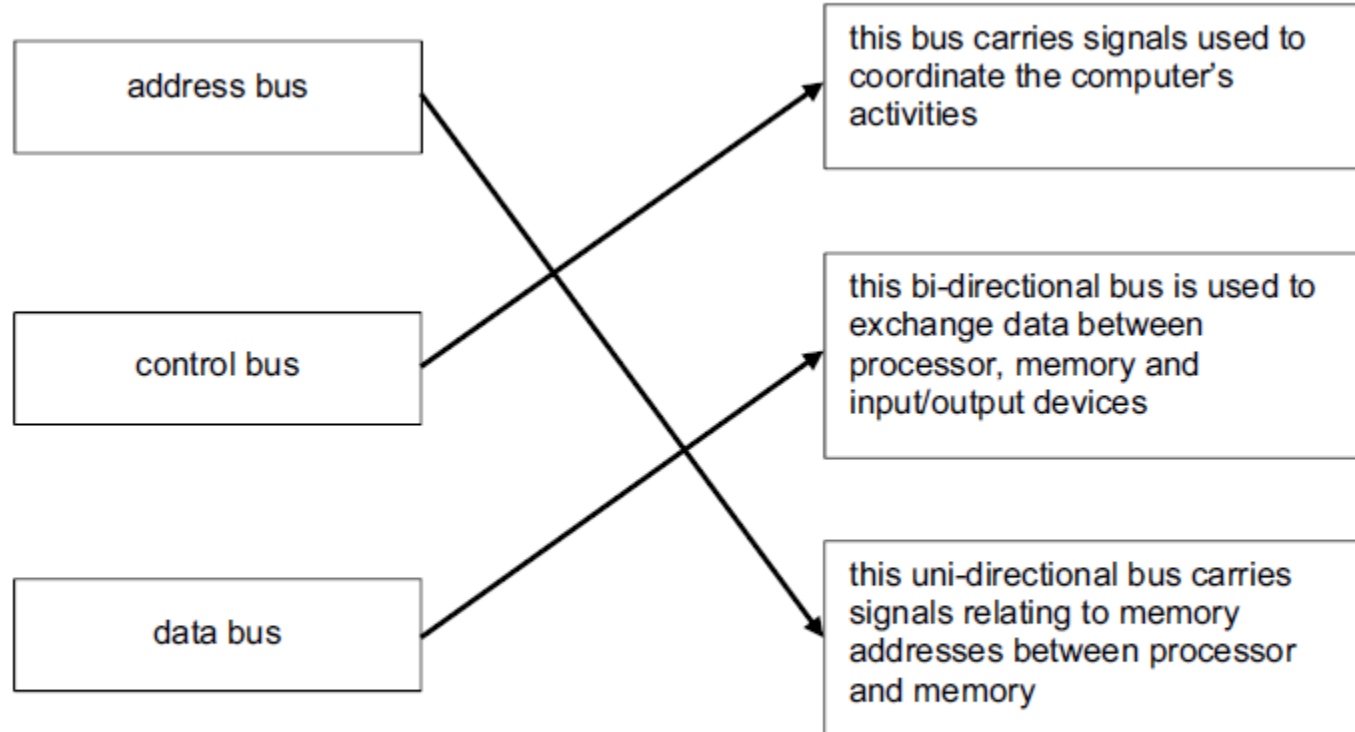
this bus carries signals used to coordinate the computer's activities

this bi-directional bus is used to exchange data between processor, memory and input/output devices

this uni-directional bus carries signals relating to memory addresses between processor and memory

IGCSE Exam Answer 2

(a)

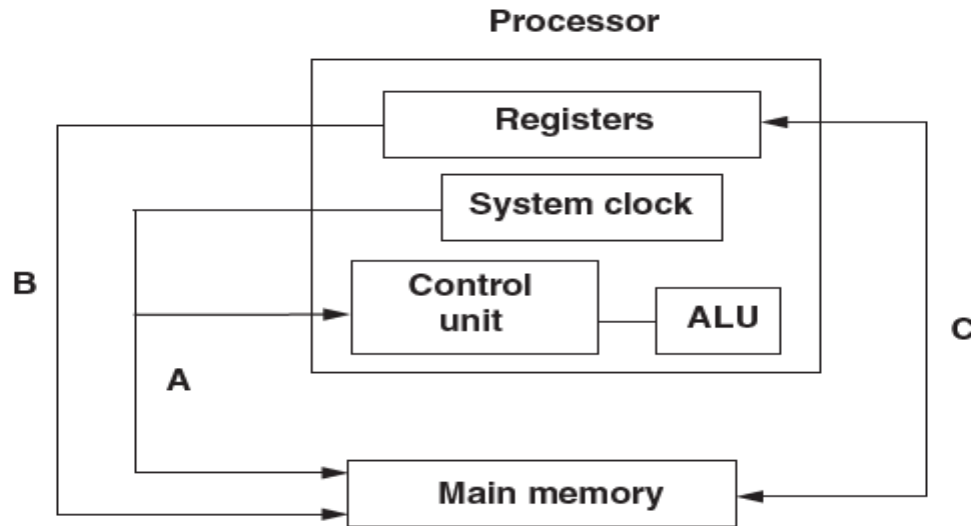


2/3 matches – 2 marks
1 match – 1 mark

[2]

IGCSE Exam Question 3

2 (a)



The diagram above shows a simplified form of processor architecture.

Name the three buses labelled A, B and C.

A

B

C

[3]

IGCSE Exam Question 3

(b) State the role of each of the following special purpose registers used in a typical processor.

Program Counter

.....

Memory Data Register

.....

Current Instruction Register

.....

Memory Address Register

.....

[4]

IGCSE Exam Answer 3

2 (a) A = control bus

B = address bus

C = data bus

[3]

(b) Program Counter – stores the address of next instruction to be executed

Memory Data Register – stores the data in transit between memory and other registers //
holds the instruction before it is passed to the CIR

Current Instruction Register – stores the current instruction being executed

Memory Address Register – stores the address of the memory location which is about to be
accessed

[4]

3.1:Computer Architecture

Watch the video for Fetch-decode-execute cycle

https://www.youtube.com/watch?v=8Qx6Dr_aKXs

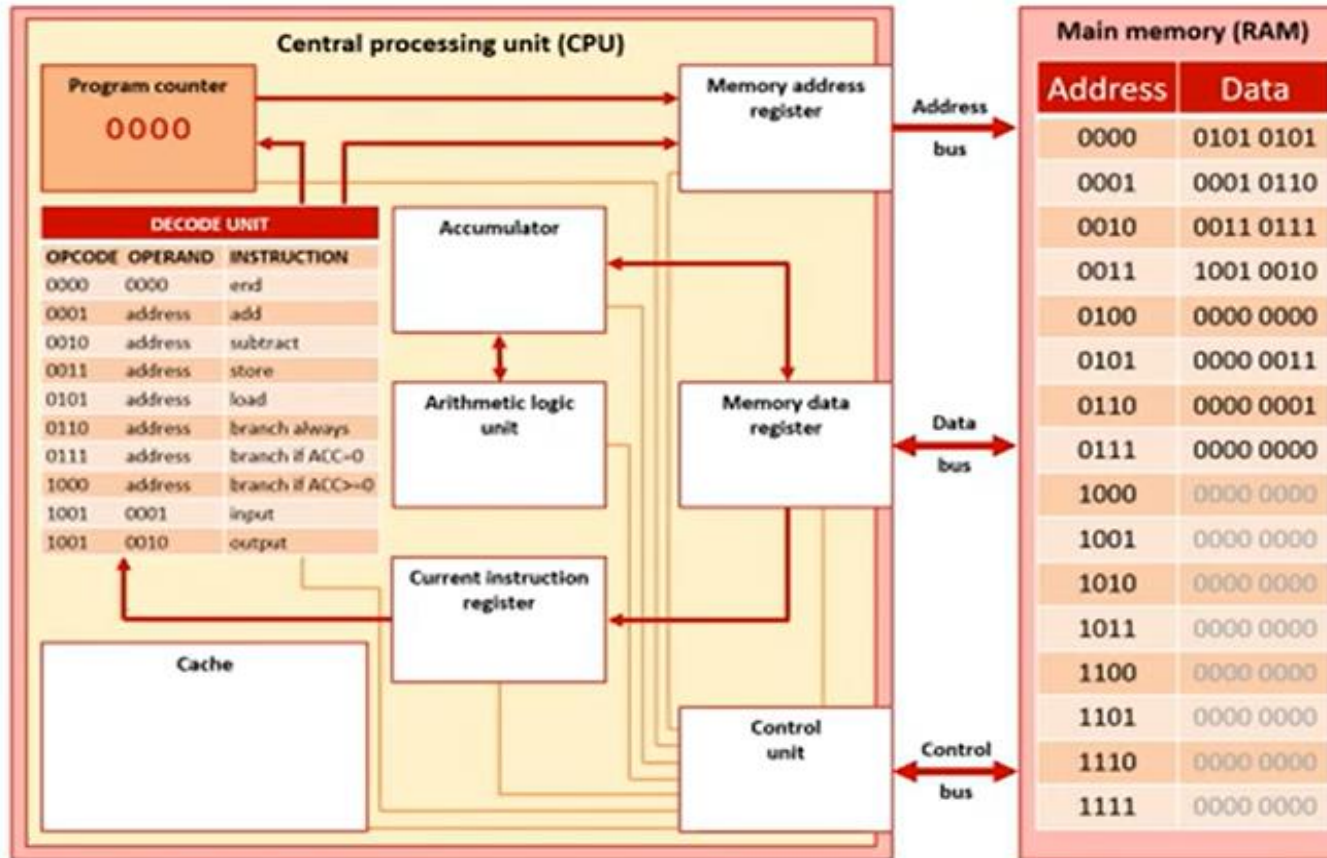


3.1:Computer Architecture

Fetch-decode-execute cycle

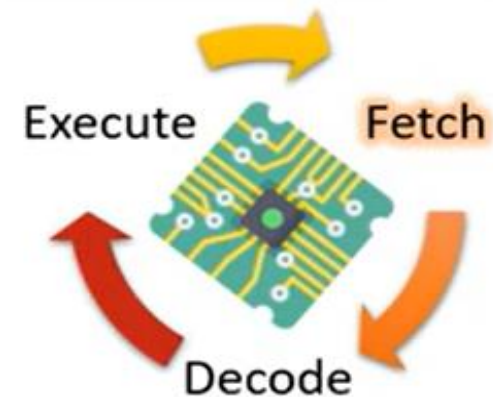
Step 1:

Fetch-decode-execute cycle



Fetch stage

The **program counter** is checked, as it holds the address of the next instruction to be executed.

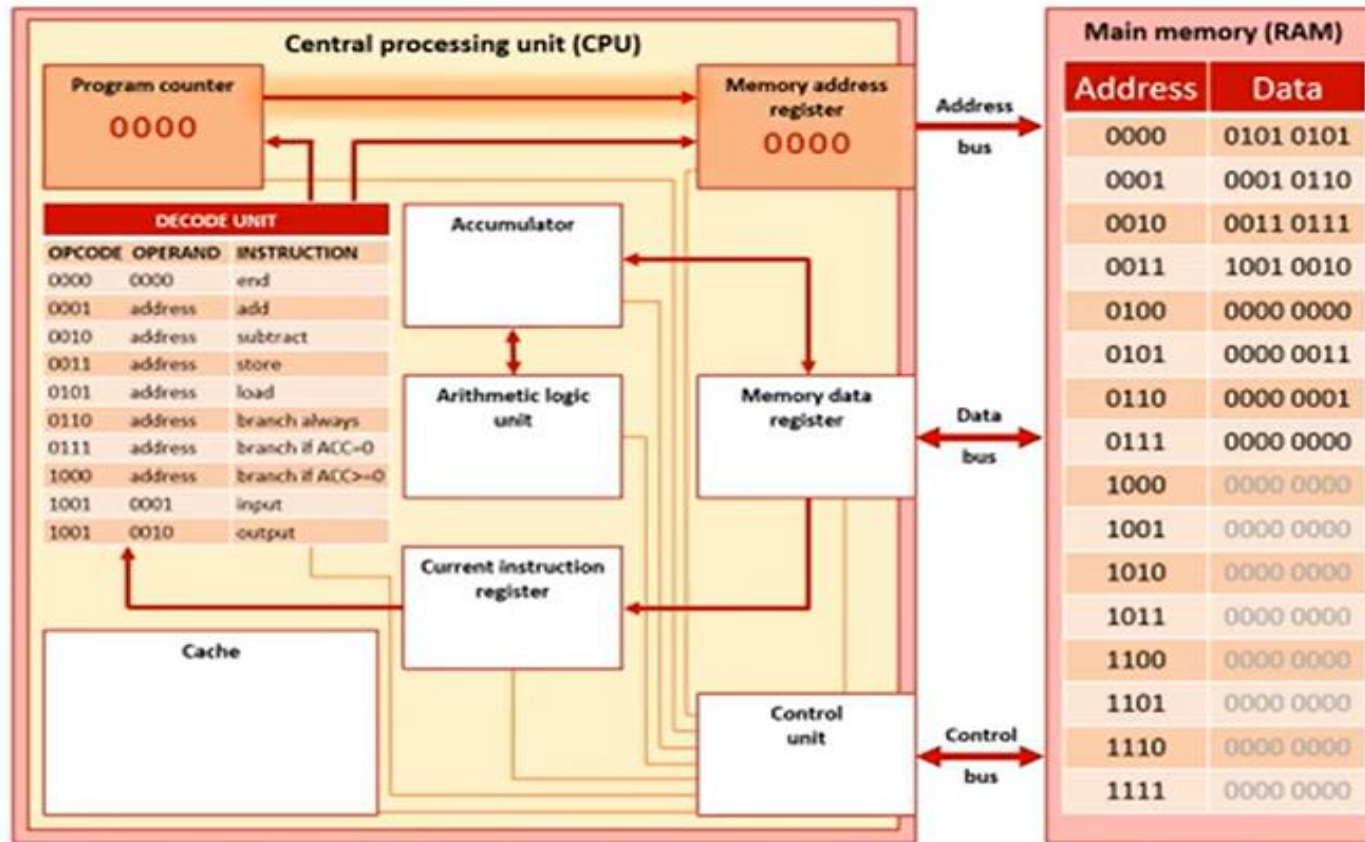


3.1:Computer Architecture

Fetch-decode-execute cycle

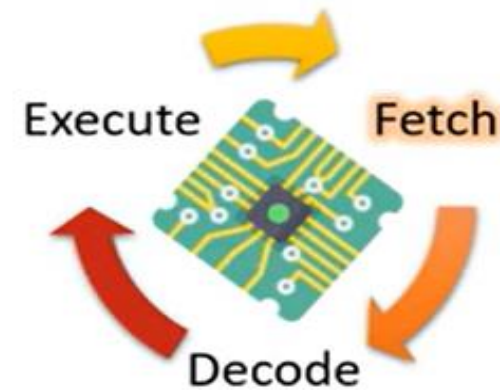
Step 2:

Fetch-decode-execute cycle



Fetch stage

The address is then copied into the memory address register (MAR).

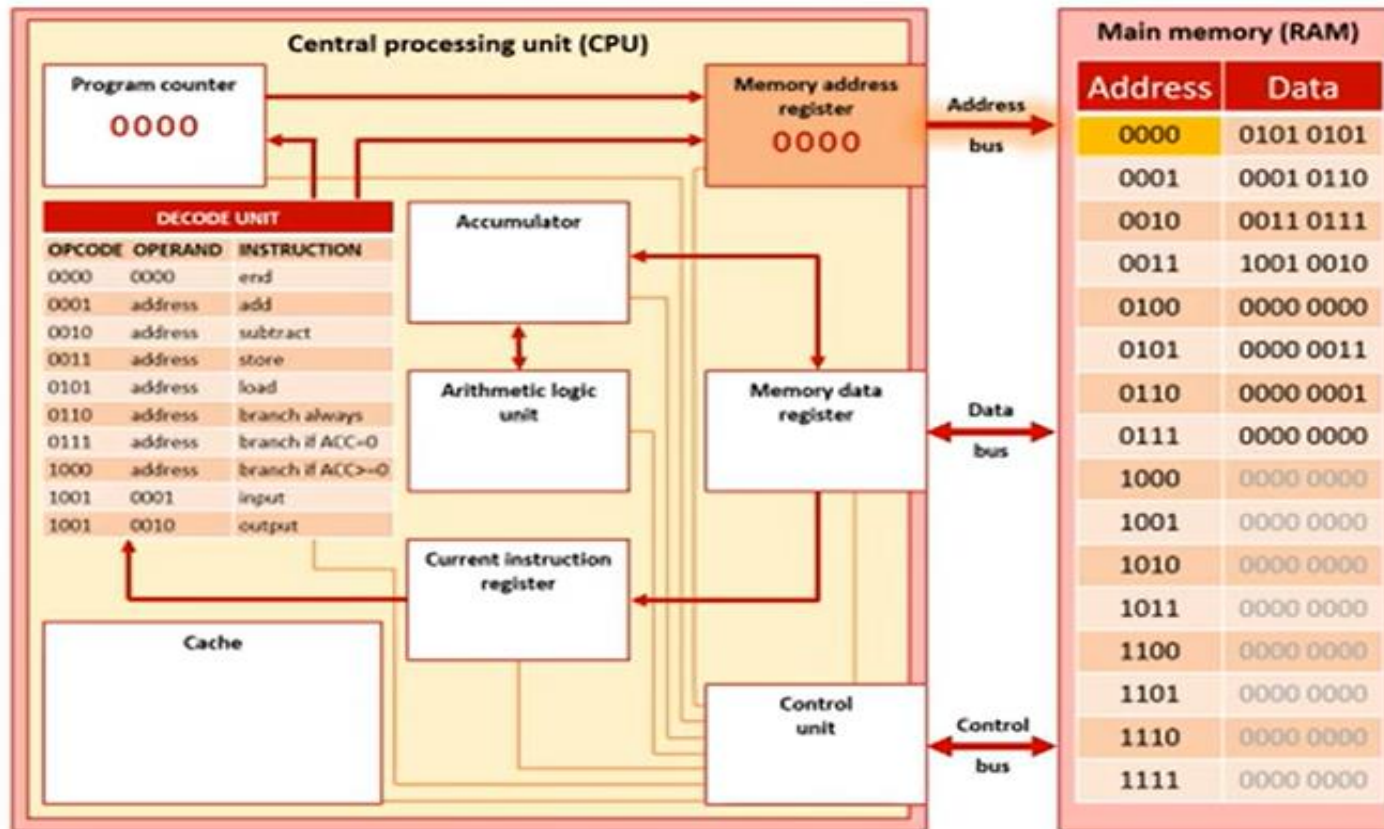


3.1:Computer Architecture

Fetch-decode-execute cycle

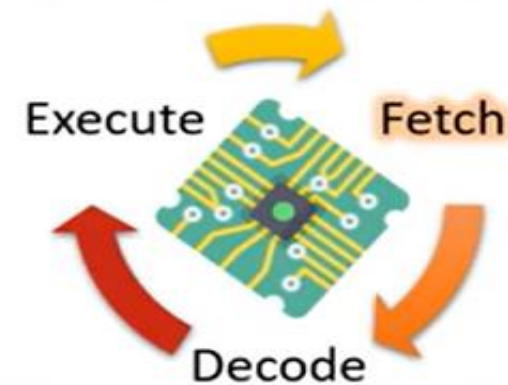
Step 3:

Fetch-decode-execute cycle



Fetch stage

The address is then sent along the **address bus** to main memory, where it waits to receive a signal from the **control bus**.

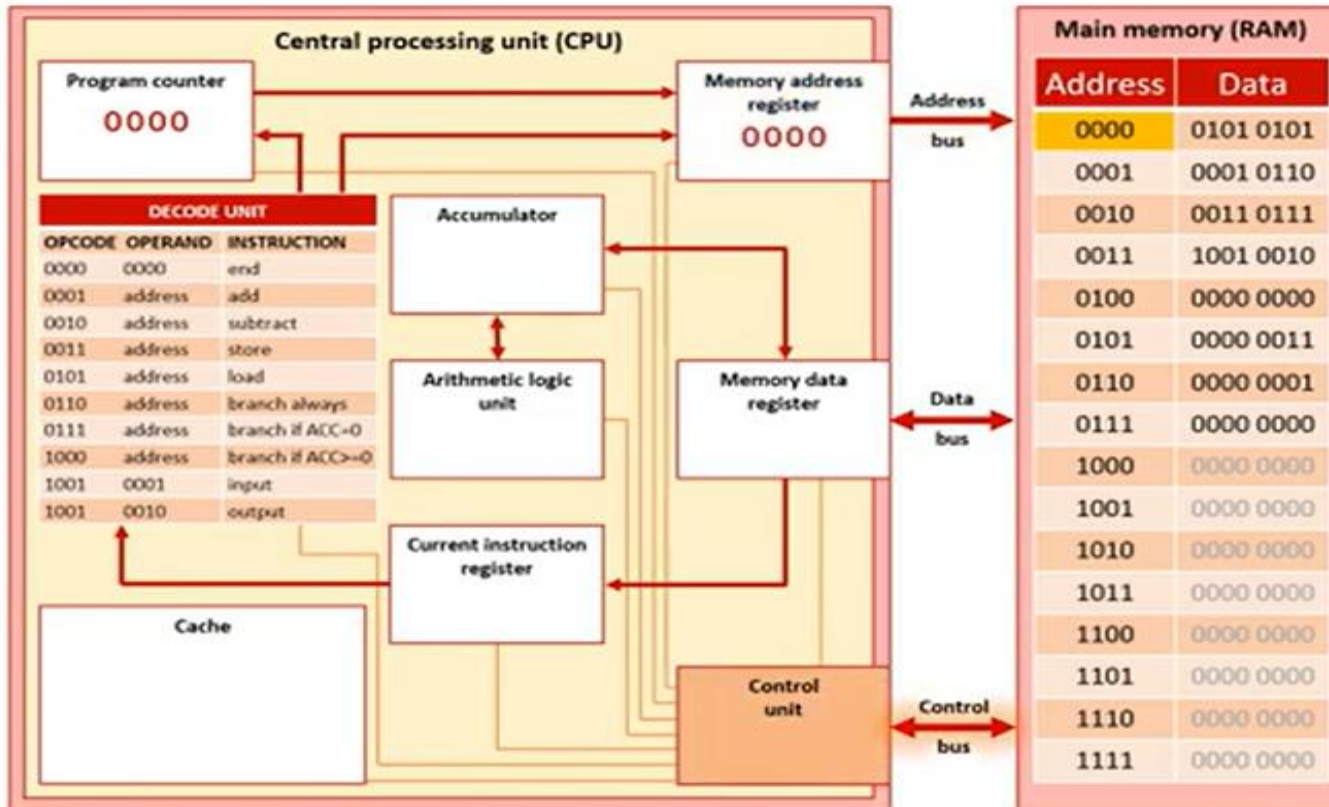


3.1:Computer Architecture

Fetch-decode-execute cycle

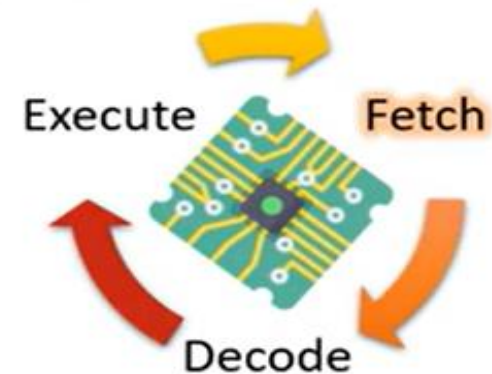
Step 4:

Fetch-decode-execute cycle



Fetch stage

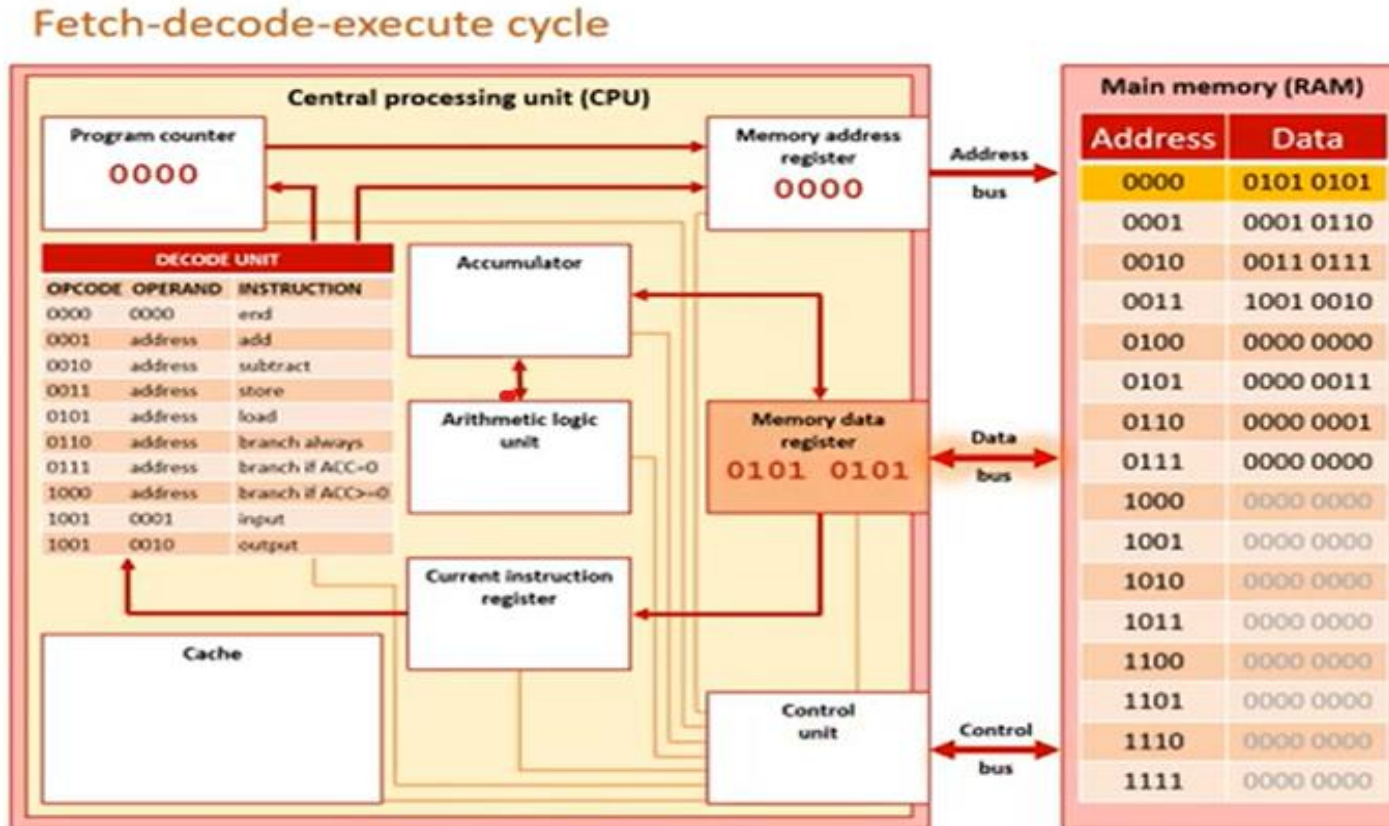
As we want to read the data stored in memory address 0000, the **control unit** sends a read signal along the **control bus** to main memory.



3.1:Computer Architecture

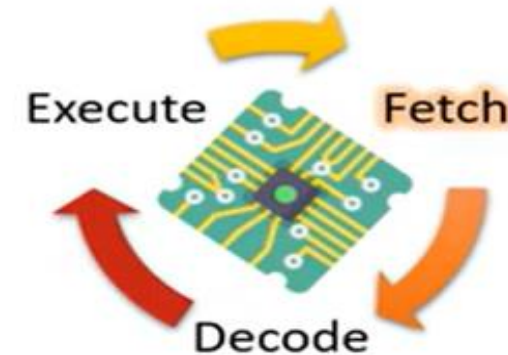
Fetch-decode-execute cycle

Step 5:



Fetch stage

The contents stored in memory at address 0000 can now be sent along the data bus to the memory data register.

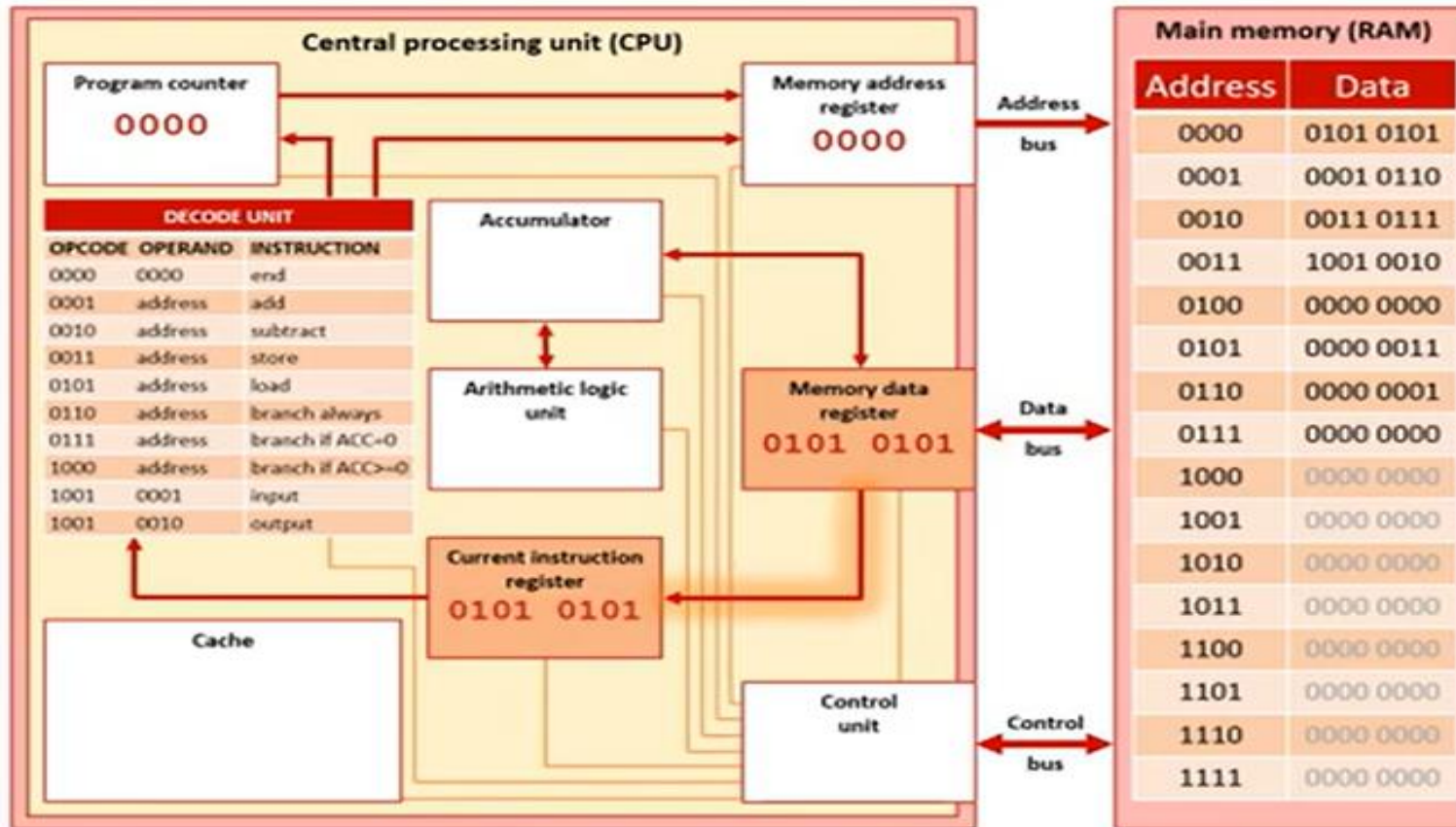


3.1:Computer Architecture

Fetch-decode-execute cycle

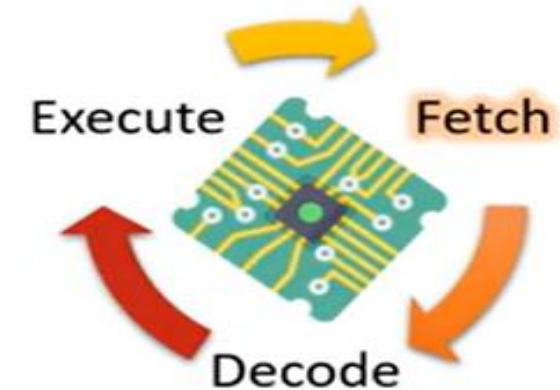
Step 6:

Fetch-decode-execute cycle



Fetch stage

As we are currently in the process of **fetching** an instruction, the data received from memory by the **memory data register** is copied into the **current instruction register**.

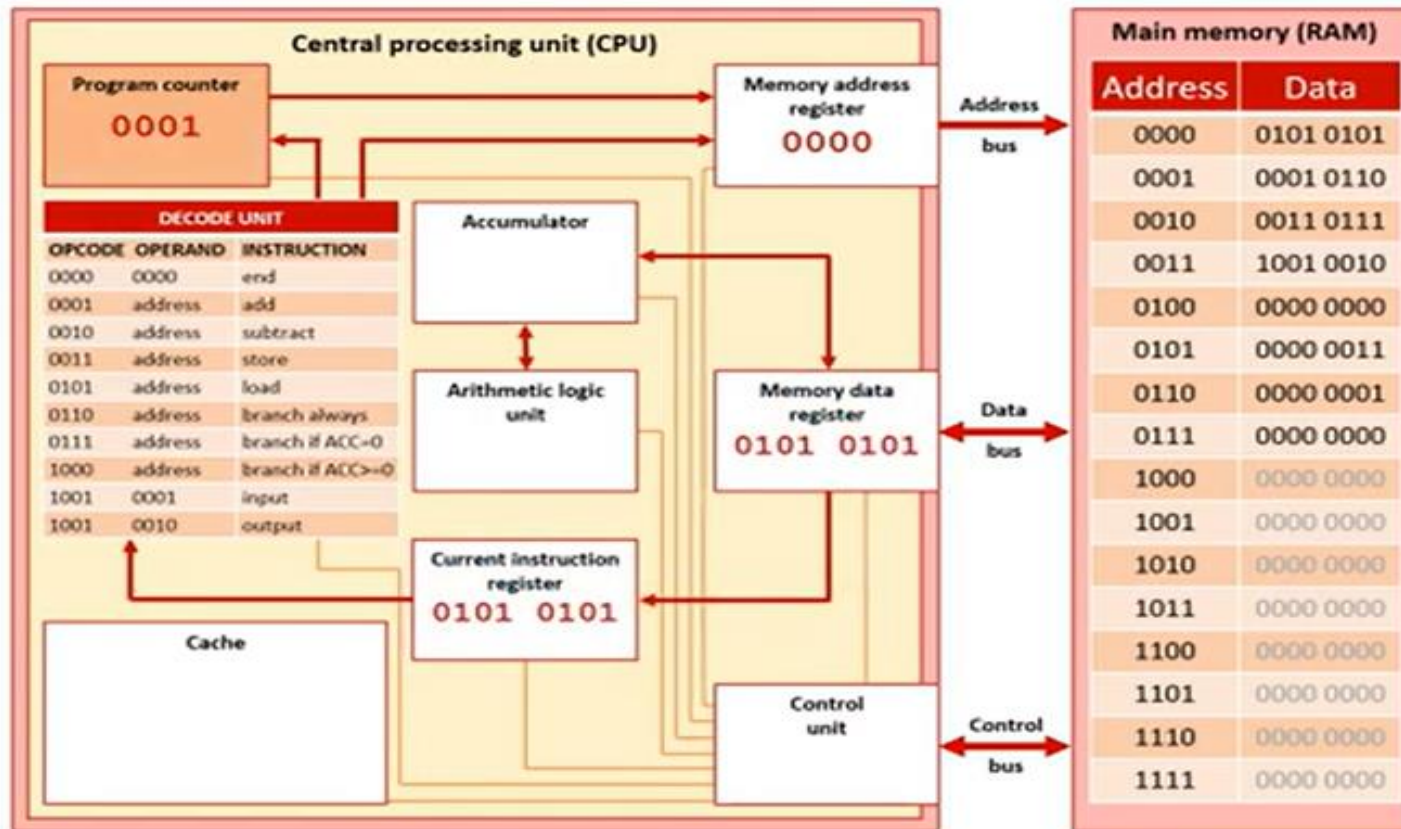


3.1:Computer Architecture

Fetch-decode-execute cycle

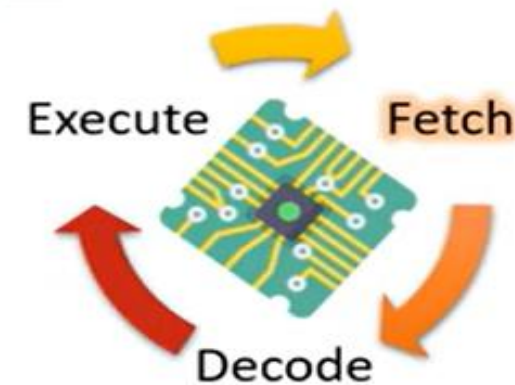
Step 7:

Fetch-decode-execute cycle



Fetch stage

The instruction has now been fetched. Before we proceed to the decode phase, we must increment the contents of the **program counter** so the address it contains points to the next instruction to be executed.

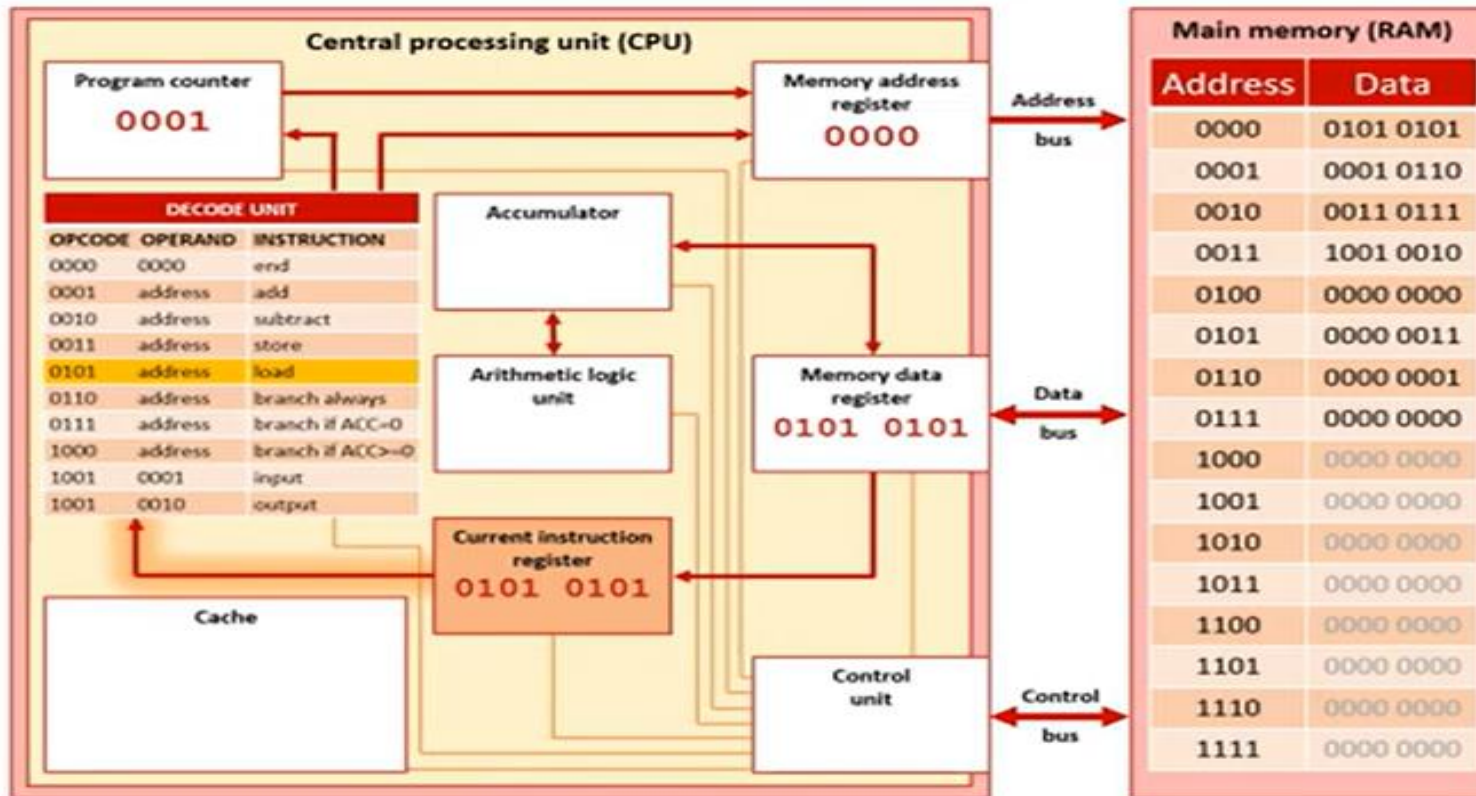


3.1:Computer Architecture

Fetch-decode-execute cycle

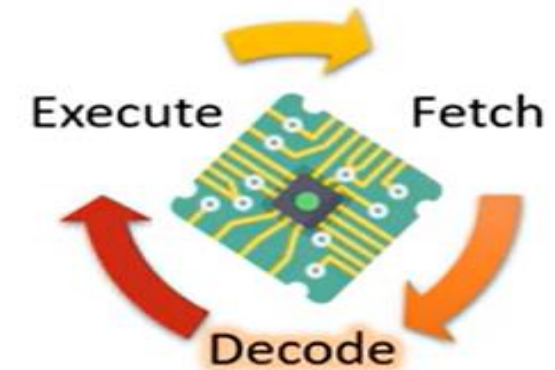
Step 8:

Fetch-decode-execute cycle



Decode stage

The instruction held in the **current instruction register** is now decoded by the decode unit.

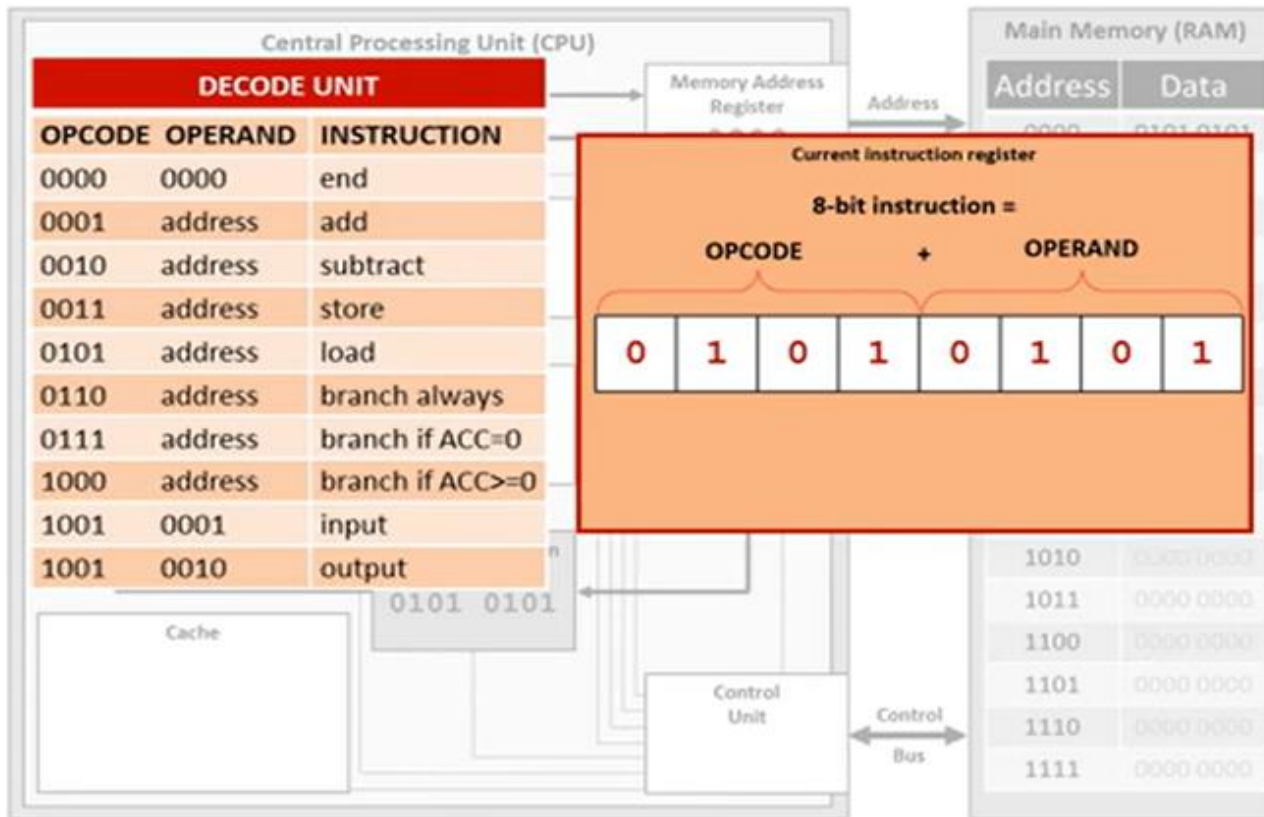


3.1:Computer Architecture

Fetch-decode-execute cycle

Step 9:

Fetch-decode-execute cycle

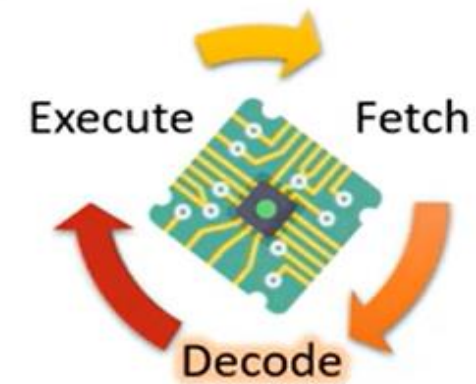


Decode stage

The instruction is made up of two parts:

- **Opcode:** What to do
- **Operand:** What to do it to

The operand could contain the actual data or the address where the data is.

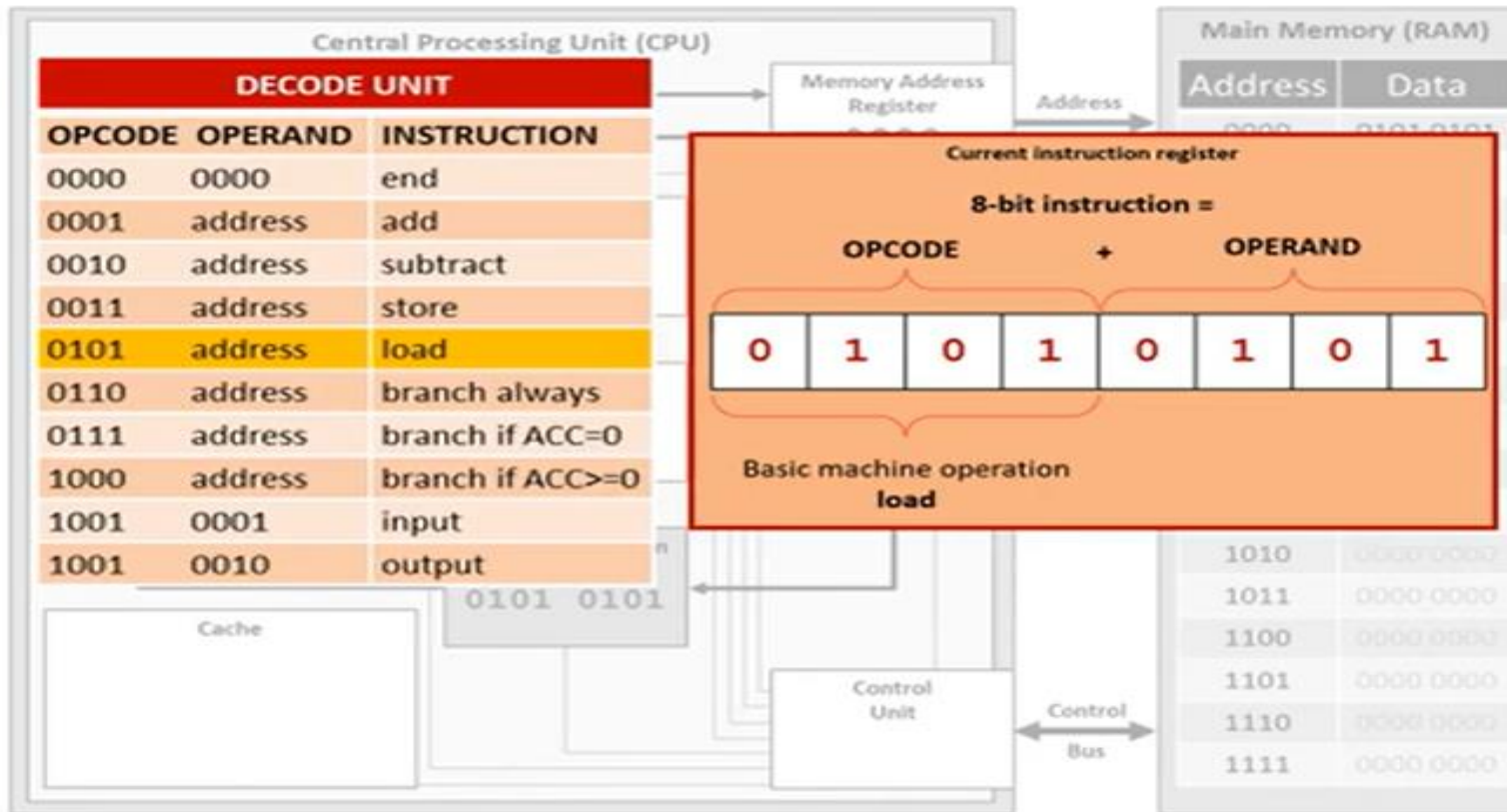


3.1:Computer Architecture

Fetch-decode-execute cycle

Step 10:

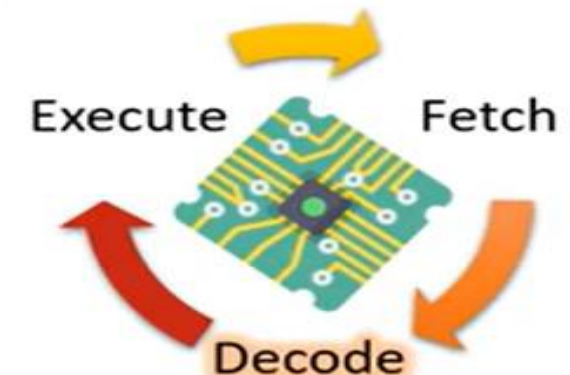
Fetch-decode-execute cycle



Decode stage

By decoding this instruction, we can see that the operation we need is a *load* operation.

We need to load contents of memory location 0101 into the CPU's **accumulator**.

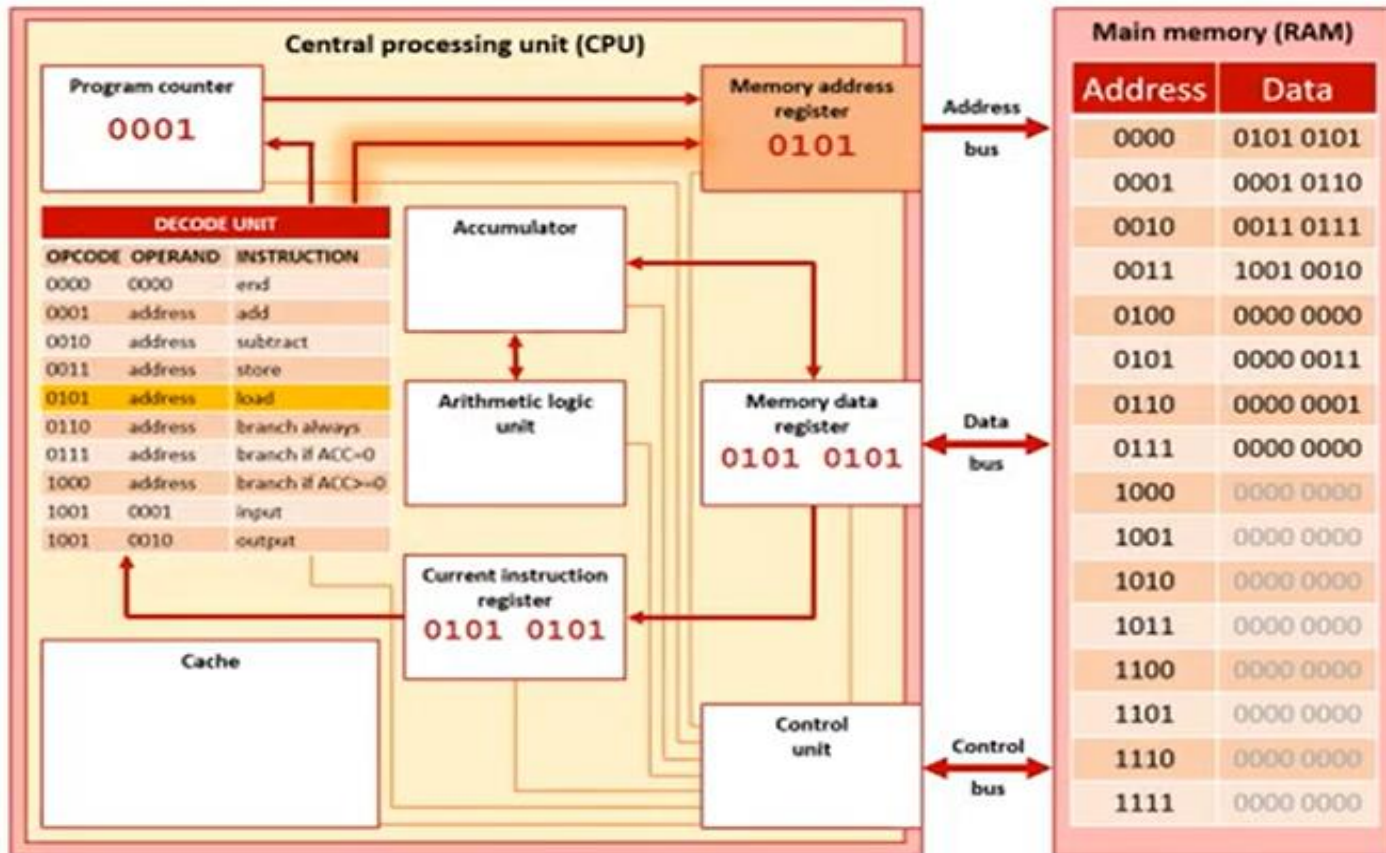


3.1:Computer Architecture

Fetch-decode-execute cycle

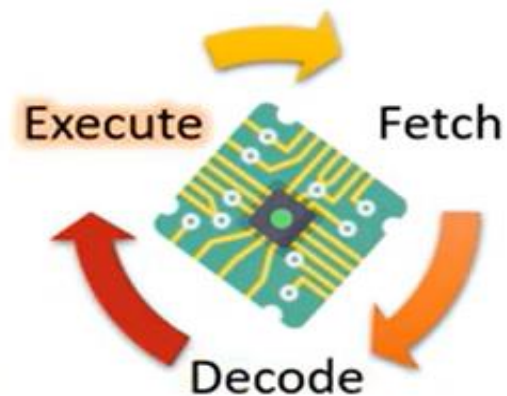
Step 11:

Fetch-decode-execute cycle



Execute stage

We now send address 0101 to the memory address register.

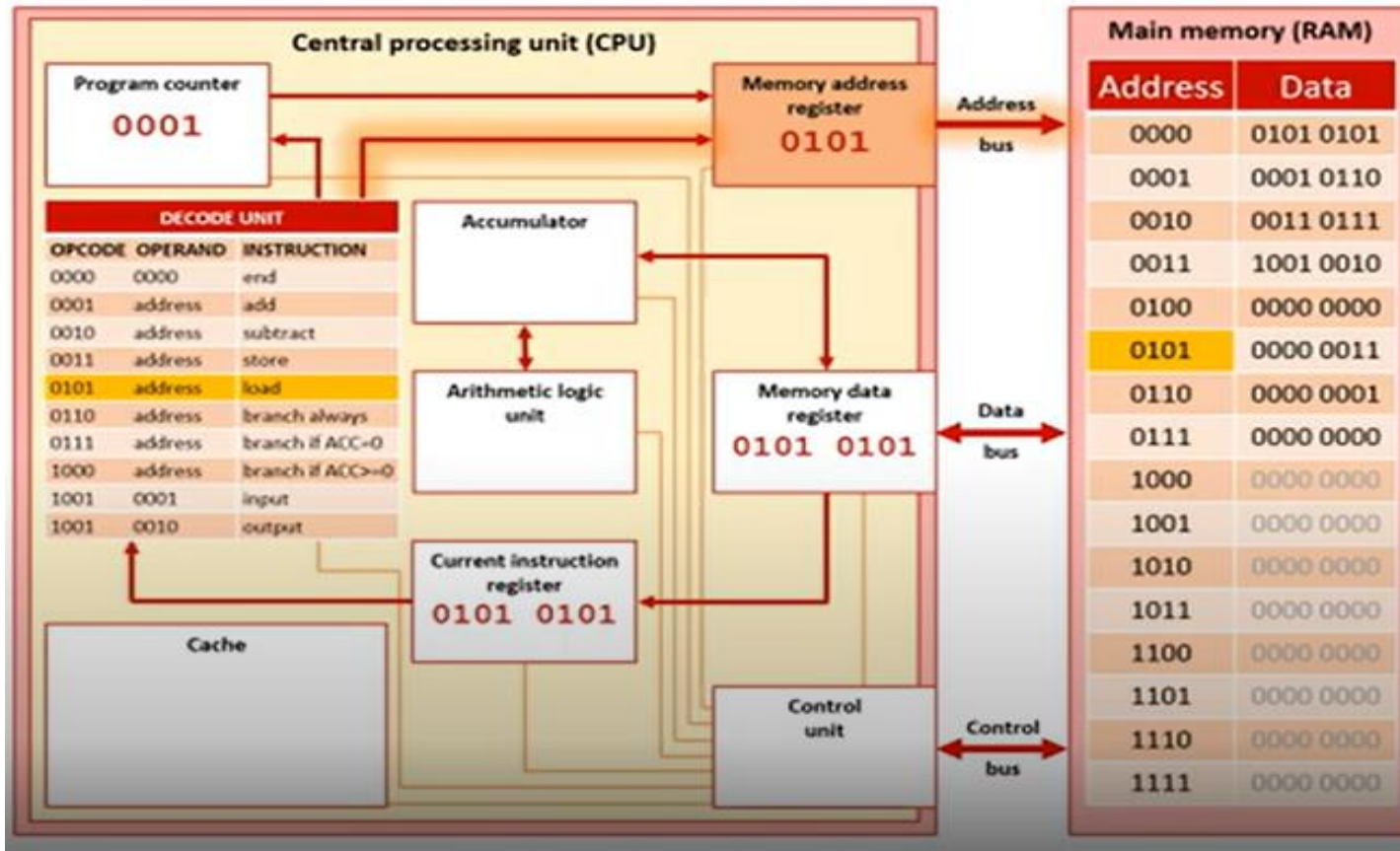


3.1:Computer Architecture

Fetch-decode-execute cycle

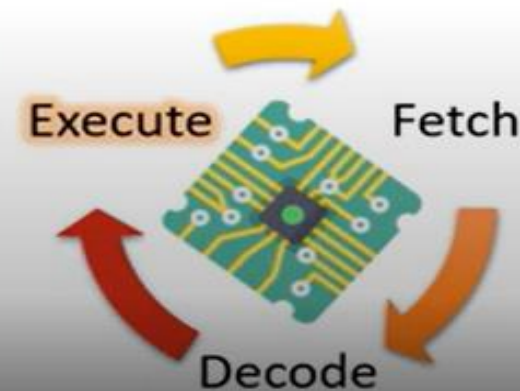
Step 12:

Fetch-decode-execute cycle



Execute stage

We send the address down the **address bus** to main memory.

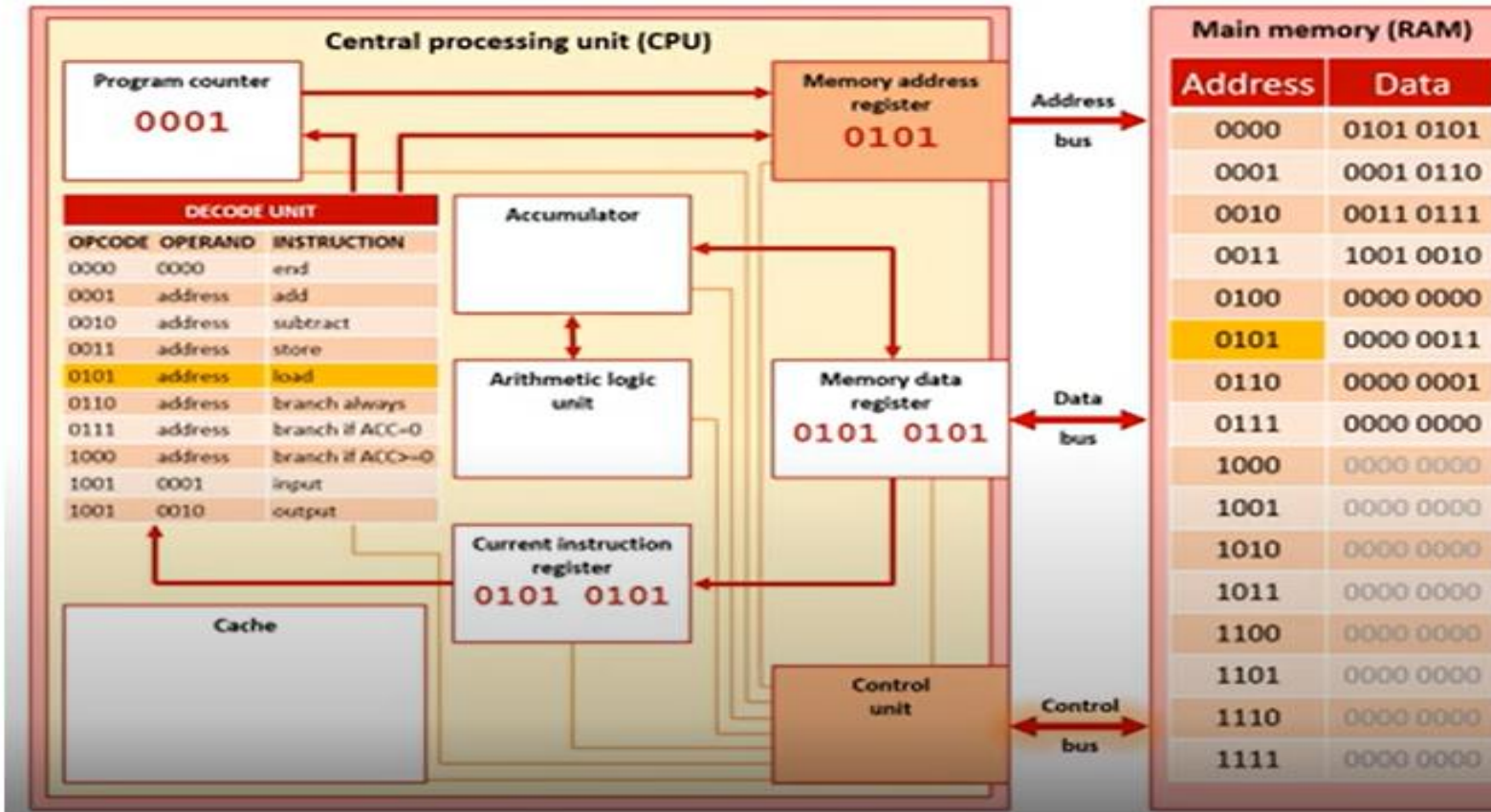


3.1:Computer Architecture

Fetch-decode-execute cycle

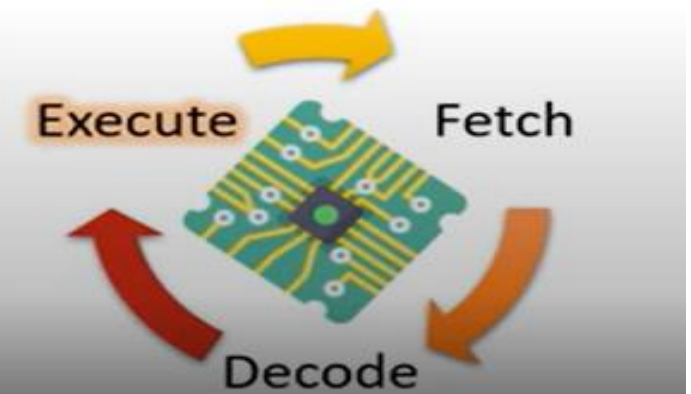
Step 13:

Fetch-decode-execute cycle



Execute stage

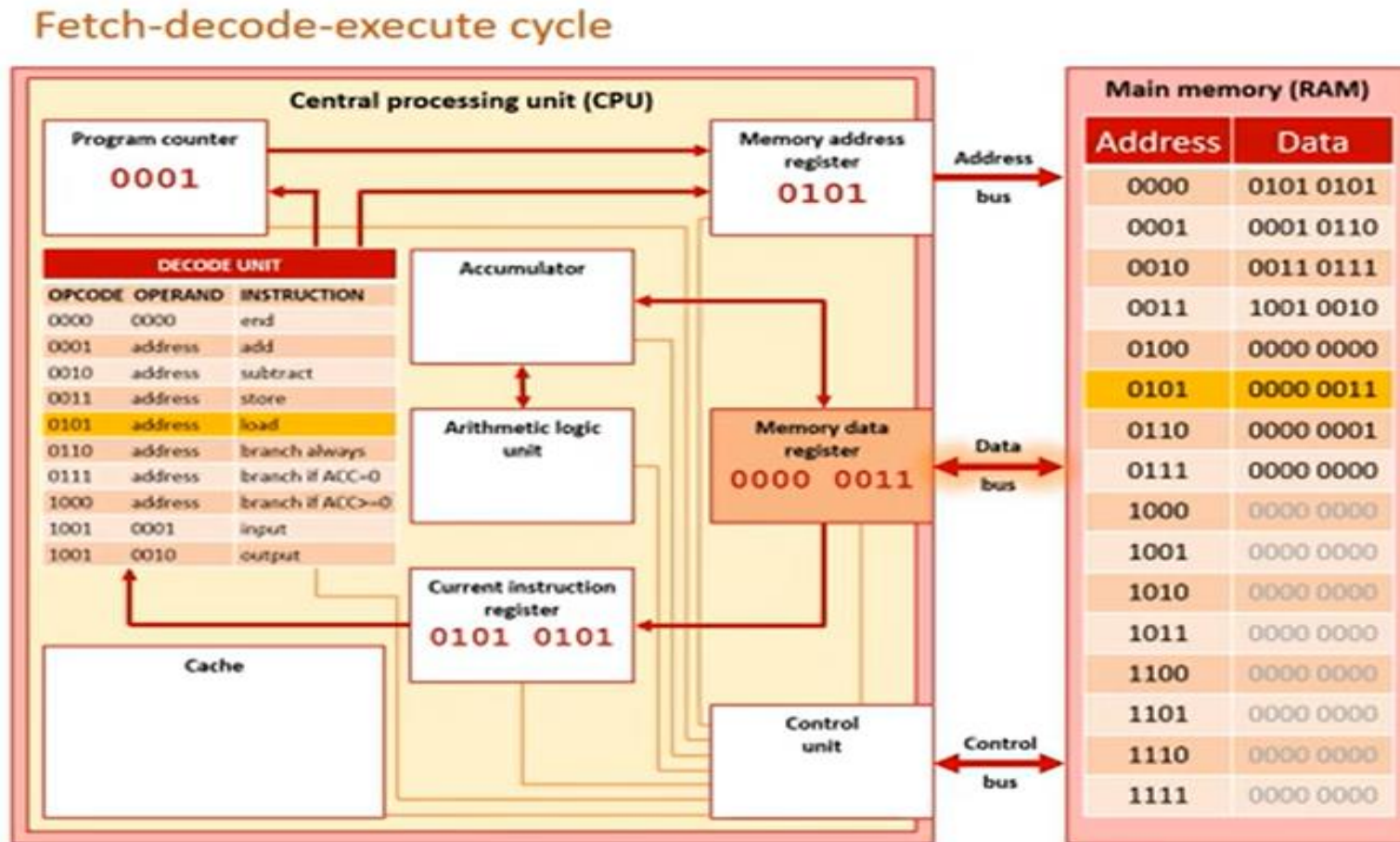
We want to read the data stored in memory address 0101, so the **control unit** sends a read signal along the **control bus** to main memory.



3.1:Computer Architecture

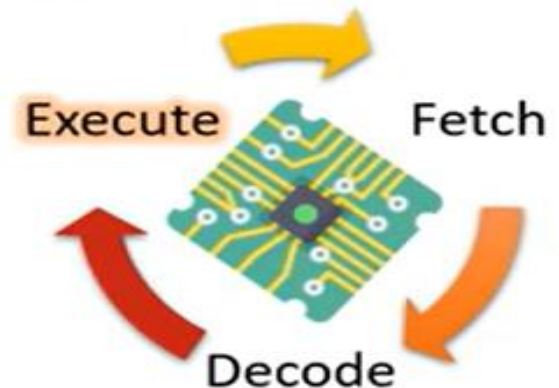
Fetch-decode-execute cycle

Step 14:



Execute stage

The contents stored in memory at address 0101 can now be sent along the data bus to the memory data register.

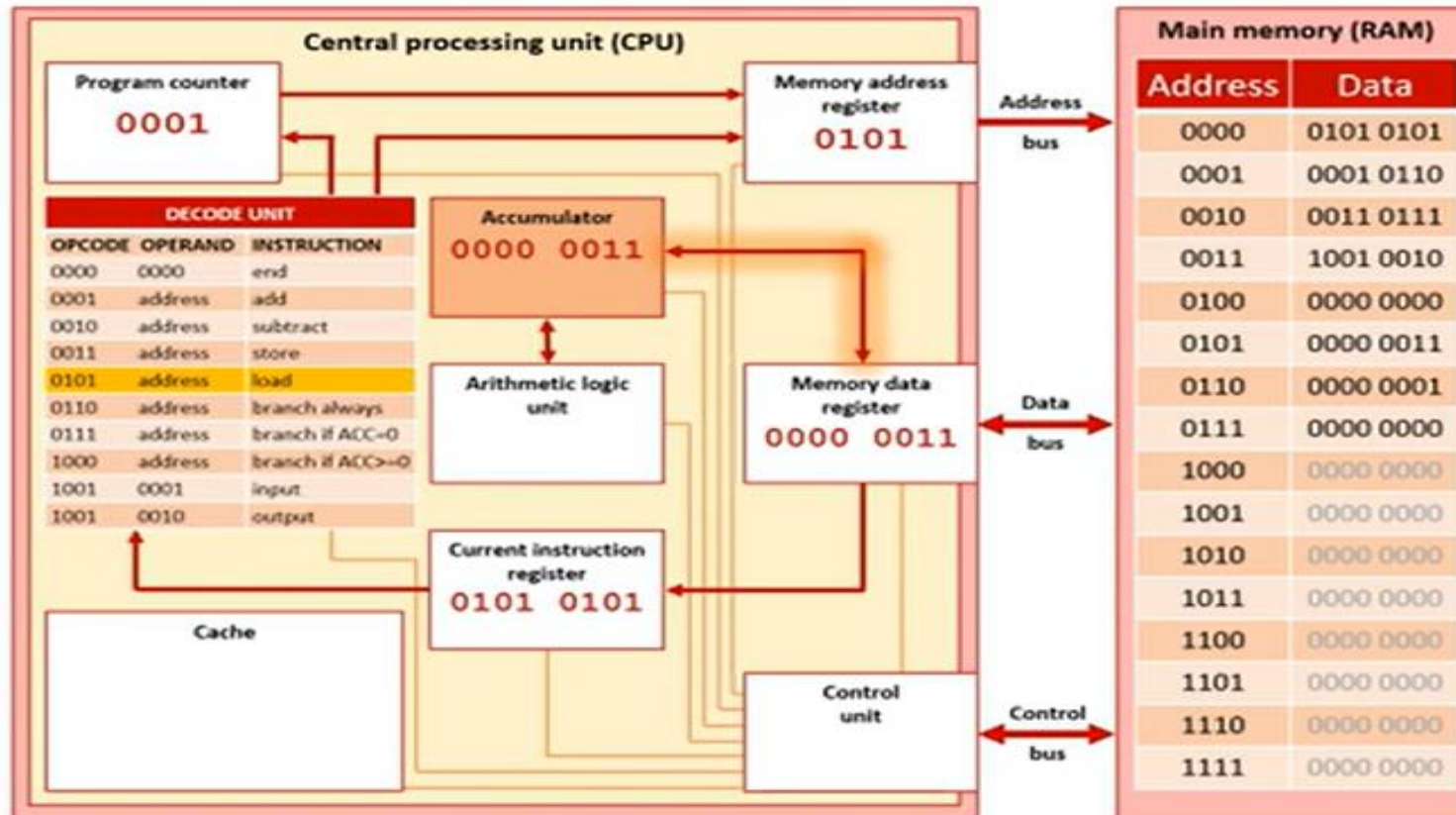


3.1:Computer Architecture

Fetch-decode-execute cycle

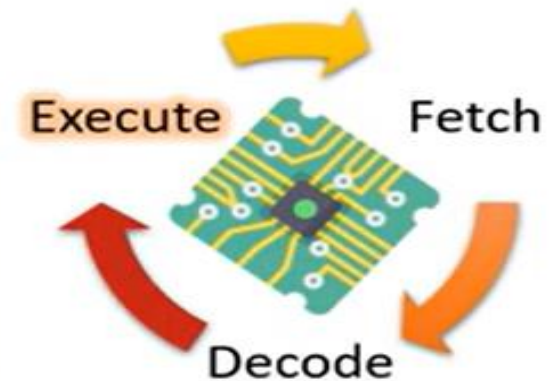
Step 15:

Fetch-decode-execute cycle



Execute stage

The contents of the **memory data register** are now copied to the **accumulator**, one of several general-purpose registers in modern CPUs. This instruction is now complete.

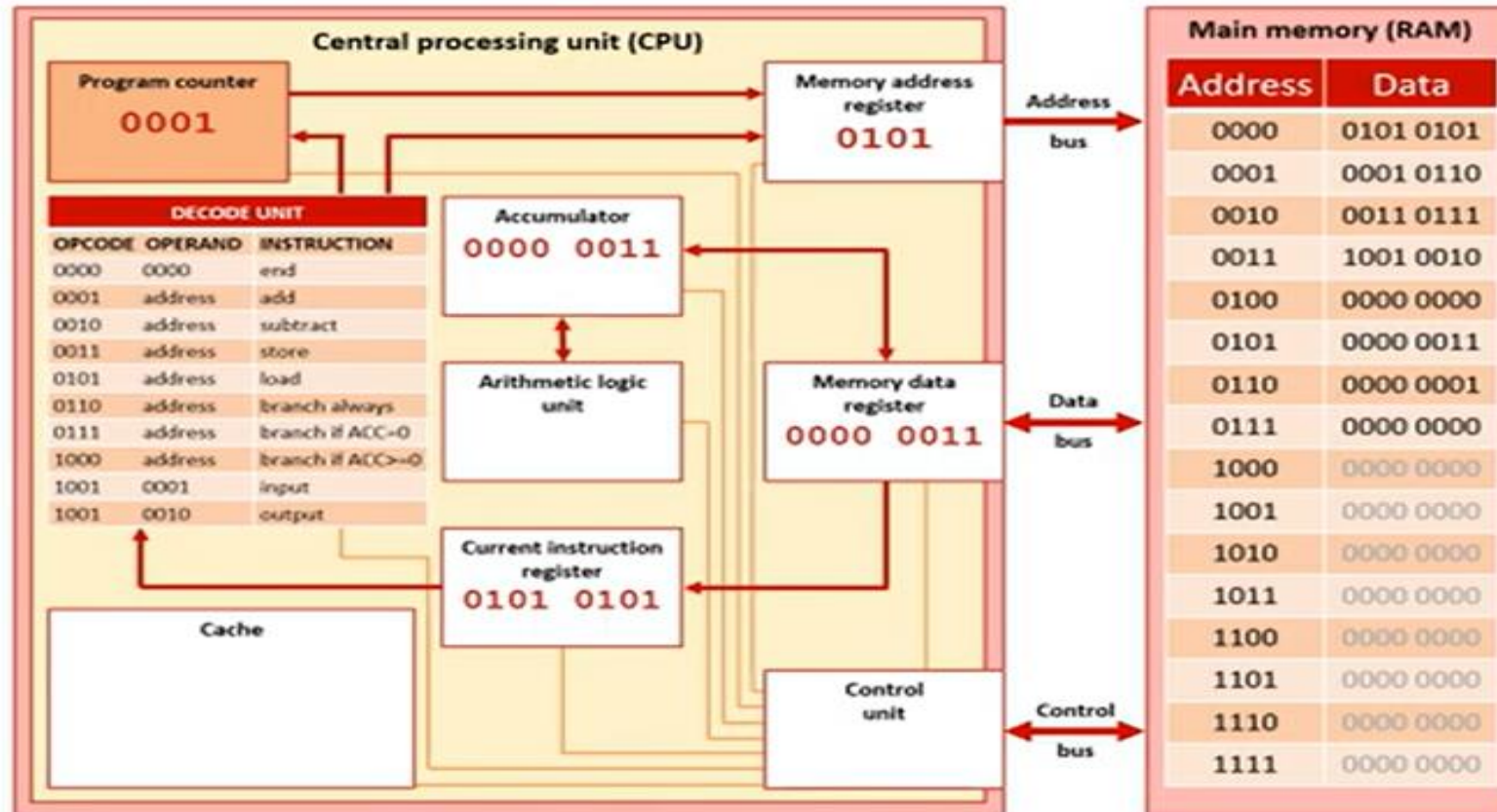


3.1:Computer Architecture

Fetch-decode-execute cycle

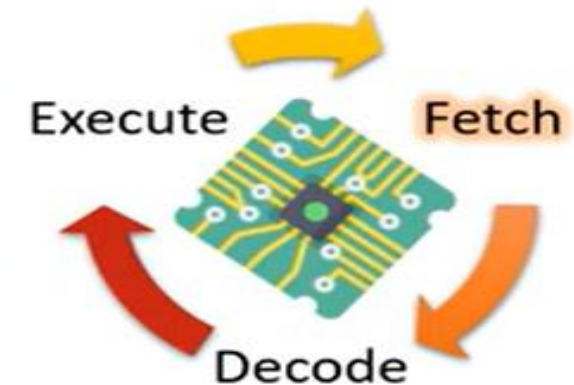
Step 16:

Fetch-decode-execute cycle



Fetch stage

The start of the brand-new fetch-decode-execute cycle begins. Once again, the **program counter** is checked for the address of the next instruction.



3.1:Computer Architecture

Fetch-decode-execute cycle

Fetch-decode-execute cycle

The purpose of the **CPU** is to **fetch**, **decode** and **execute** instructions – it does this billions of times a second.



Fetch

1. The **program counter** is checked for the address of the next instruction to be executed.
2. The contents of the **program counter** are copied into the **memory address register**.
3. The **program counter** is incremented.
4. The **control unit** sends a memory read request to **main memory** down the **control bus**.
5. The address is sent down the **address bus** from the **memory address register** to **main memory**.
6. The contents of **main memory** are sent down the **data bus** into the **memory data register**.
7. The contents of the **memory data register** are copied into the **current instruction register**.

Decode

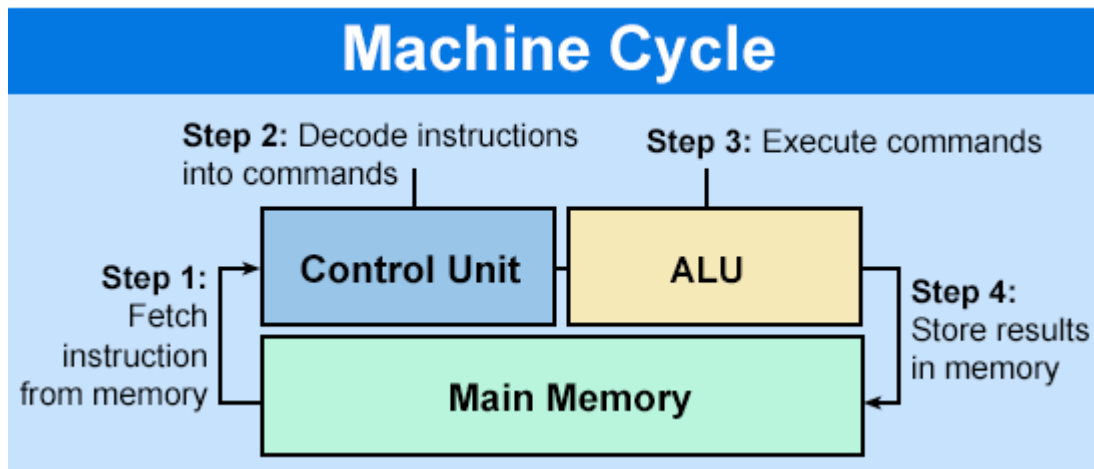
8. The instruction in the **current instruction register** is inspected and **decoded** to work out what needs to be done.

Execute

9. The instruction is carried out – this could be:
 - a) Going back to **main memory** and **fetching** data.
 - b) Performing a calculation.
 - c) Storing information back in **main memory**.
10. Depending on the instruction, data may end up being copied into the **ALU** and/or the **accumulator**.

The fetch–execute cycle

- **Fetch** - Retrieve an [instruction](#) from [memory](#).
- **Decode** - Translate the retrieved instruction into a series of computer commands.
- **Execute** - Execute the computer commands.
- **Store** - Send and write the results back in memory.



Fetch mean: to go to another place to get something

The fetch–execute cycle

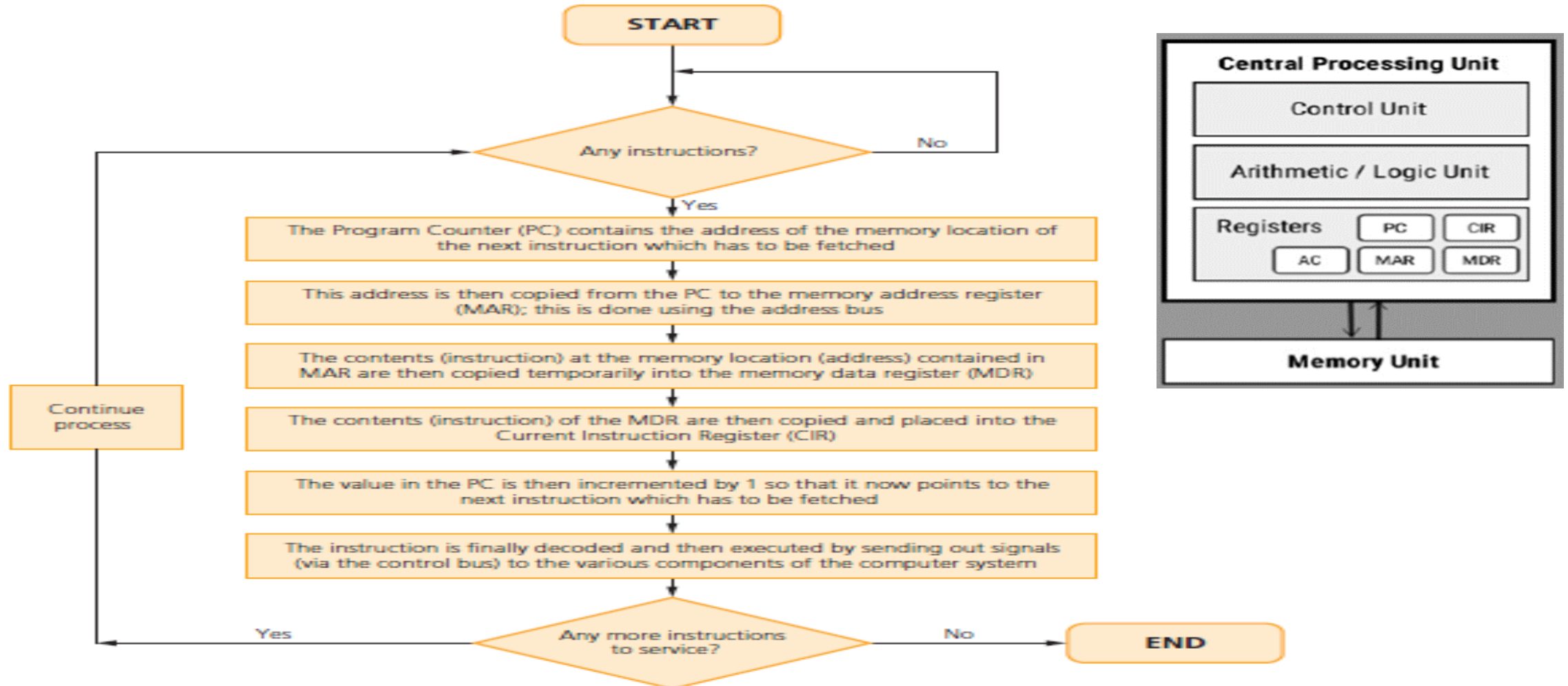
Example of a machine cycle

Below is an example of a machine cycle performing the steps mentioned above for a math problem.

- The computer user enters a math problem that's stored in memory. The computer fetches that instruction from memory.
- The control unit decodes that math problem into instructions the computer understands.
- The ALU executes the instructions to get the answer to the math problem.
- The problem's answer is sent to and stored in memory where it can be accessed and displayed to the user.

The fetch–execute cycle

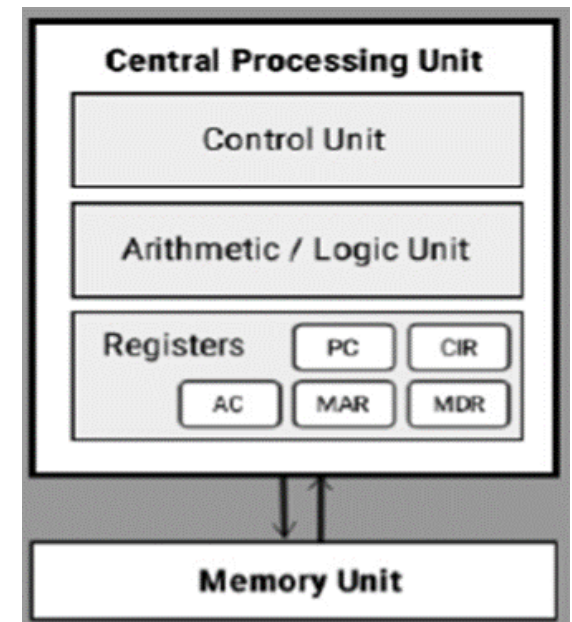
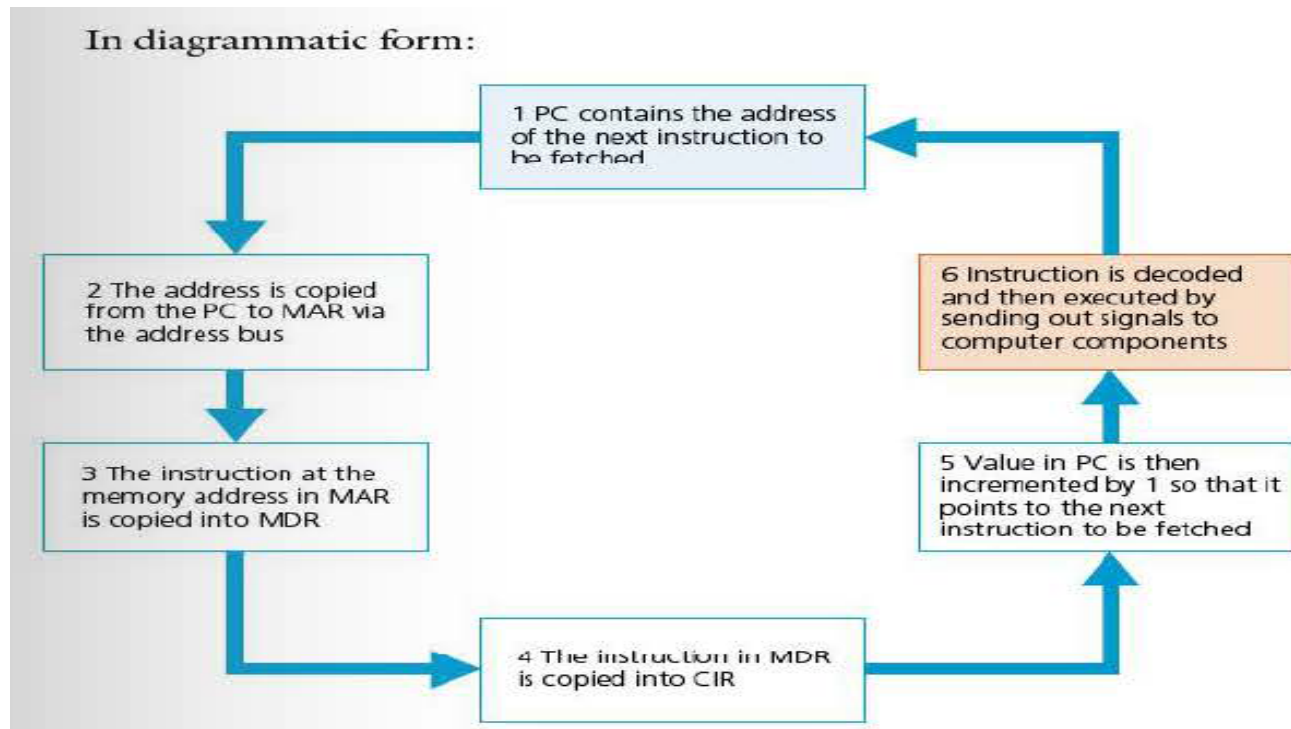
Figure 3.3 shows how the Fetch–Decode–Execute cycle is carried out in the von Neumann computer model.

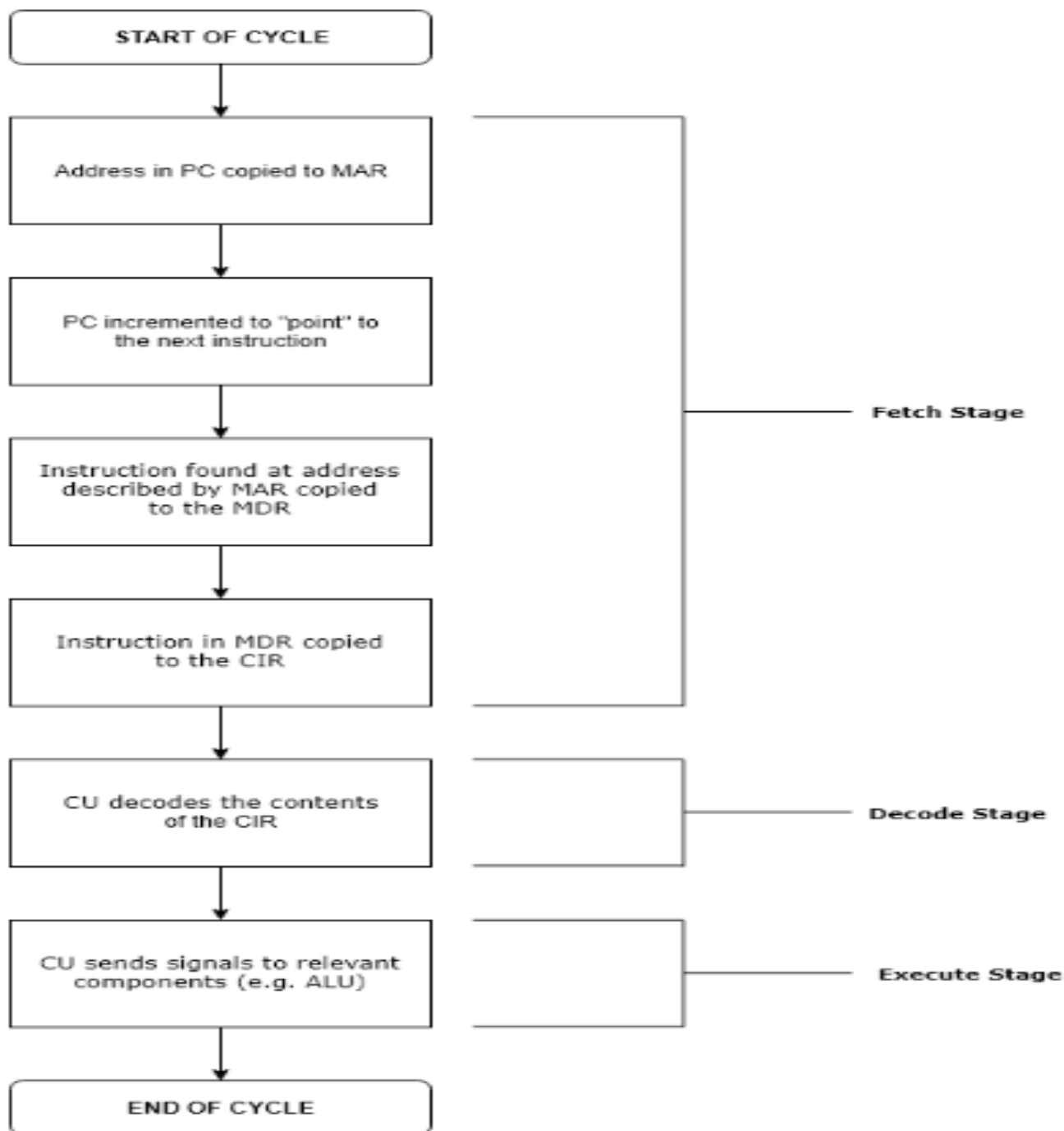


▲ Figure 3.3 Fetch–Decode–Execute cycle flowchart

The fetch–execute cycle

- Figure below shows how the fetch–execute cycle is carried out in the von Neumann computer model.

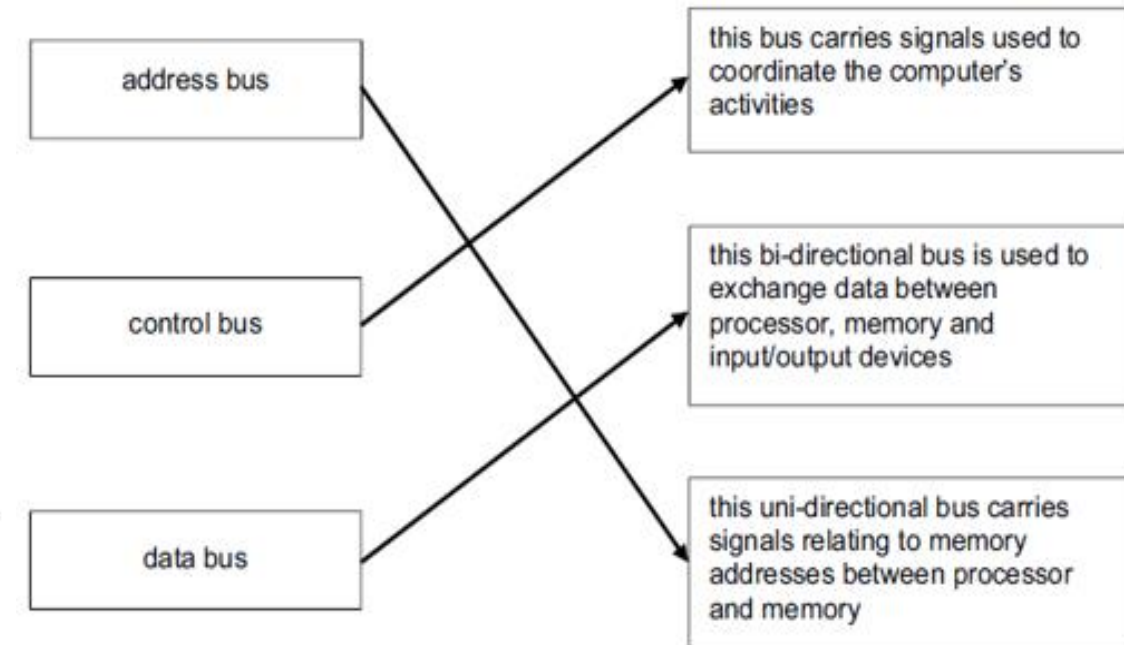




Registers

Registers are high speed storage areas in the CPU. All data must be stored in a register before it can be processed.

MAR	Memory Address Register	Holds the memory location of data that needs to be accessed
MDR	Memory Data Register	Holds data that is being transferred to or from memory
AC	Accumulator	Where intermediate arithmetic and logic results are stored
PC	Program Counter	Contains the address of the next instruction to be executed
CIR	Current Instruction Register	Contains the current instruction during processing



Control Unit

- Responsible for decoding the instructions

Main Memory (RAM)
Random Access Memory
Volatile as loses information when switched OFF.

Stage	Order
address is copied from PC to MAR; this is done using the address bus	2
contents at memory location contained in MAR are copied temporarily into MDR	3
contents of MDR are copied and placed in CIR	4
instruction is finally decoded and then executed by sending out signals (via the control bus) to various components of the computer system	6
program counter (PC) contains address of memory location of next instruction to be fetched	1
value in PC is incremented by 1 so that it now points to the next instruction to be fetched	5

Address Bus - asks for the instruction from RAM

Data Bus - brings the instruction back

Fetch

The next instruction is fetched (got) from the main memory (RAM)

Decode

The instruction is interpreted/decoded. Works out what the instruction is.

Execute

The instructions are executed (carried out)

Central Processing Unit

Control Unit

Arithmetic / Logic Unit

Registers
PC CIR
AC MAR MDR

Memory Unit

(CPU) - Central Processing Unit

This cycle runs inside the CPU continuously

Accumulator: Stores the results of calculations made by the ALU.

Quizzes - Von Neumann Architecture

<https://quizizz.com/admin/quiz/5d7a7ed8bc1488001aa21a80/von-neumann-architecture>

IGCSE Exam Question 1

11

(b) The seven stages in a von Neumann fetch-execute cycle are shown in the table below.

Put each stage in the correct sequence by writing the numbers 1 to 7 in the right hand column.
The first one has been done for you.

Stage	Sequence number
the instruction is then copied from the memory location contained in the MAR (memory address register) and is placed in the MDR (memory data register)	
the instruction is finally decoded and is then executed	
the PC (program counter) contains the address of the next instruction to be fetched	1
the entire instruction is then copied from the MDR (memory data register) and placed in the CIR (current instruction register)	
the address contained in the PC (program counter) is copied to the MAR (memory address register) via the address bus	
the address part of the instruction, if any, is placed in the MAR (memory address register)	
the value in the PC (program counter) is then incremented so that it points to the next instruction to be fetched	

[6]

IGCSE Exam Answer 1

description of stage	sequence number
the instruction is then copied from the memory location contained in the MAR (memory address register) and is placed in the MDR (memory data register)	3
the instruction is finally decoded and is then executed	7
<i>the PC (program counter) contains the address of the next instruction to be fetched</i>	(1)
the entire instruction is then copied from the MDR (memory data register) and placed in the CIR (current instruction register)	4
the address contained in the PC (program counter) is copied to the MAR (memory address register) via the address bus	2
the address part of the instruction is placed in the MAR (memory address register)	6
the value in the PC (program counter) is then incremented so that it points to the next instruction to be fetched	5*

The incrementation of the program counter can appear at any stage after 2. All other stages must be in the correct given order.

[6]

IGCSE Exam Question 2

4 a (i) Describe the function of the ALU.

(ii) Describe the function of the Control Unit. [4 marks]

b The following table shows the stages in the fetch–execute cycle. They are not written in the correct order. Place the numbers 1 to 6 in the right-hand column to show each stage in its correct order.

The first one has been done for you. [5 marks]

Stage	Order
address is copied from PC to MAR; this is done using the address bus	
contents at memory location contained in MAR are copied temporarily into MDR	
contents of MDR are copied and placed in CIR	
instruction is finally decoded and then executed by sending out signals (via the control bus) to various components of the computer system	
program counter (PC) contains address of memory location of next instruction to be fetched	1
value in PC is incremented by 1 so that it now points to the next instruction to be fetched	

IGCSE Exam Answer 2

- 4 a (i)**
- carries out arithmetic operations
 - carries out logic operations
- (ii)**
- controls operation of memory, processor and I/O devices
 - sends signals to coordinate all computer components

b

Stage	Order
address is copied from PC to MAR; this is done using the address bus	2
contents at memory location contained in MAR are copied temporarily into MDR	3
contents of MDR are copied and placed in CIR	4
instruction is finally decoded and then executed by sending out signals (via the control bus) to various components of the computer system	6
program counter (PC) contains address of memory location of next instruction to be fetched	1
value in PC is incremented by 1 so that it now points to the next instruction to be fetched	5

IGCSE Exam Question 3

A Von Neumann model for a computer system has a central processing unit (CPU) that makes use of registers.

- (a) Identify **three** registers that may be used.

Register 1

Register 2

Register 3

[3]

- (b) The CPU is responsible for processing instructions.

One stage of processing instructions is the decode stage.

- (i) Identify the **two other** stages of processing instructions.

Stage 1

Stage 2

[2]

- (ii) Identify the component of the CPU that is responsible for decoding instructions.

..... [1]

IGCSE Exam Answer 3

Question	Answer	Marks
1(a)	Any three from: – MAR – MDR // MBR – PC // IAR // NIR // SCR – ACC – CIR // IR – IAS	3
1(b)(i)	– Fetch – Execute	2
1(b)(ii)	– Control unit	1

IGCSE Exam Question 4

The fetch-execute cycle make use of registers.

(a) Describe the role of the Program Counter (PC).

.....

.....

.....

.....[2]

(b) Describe the role of the Memory Data Register (MDR).

.....

.....

.....

.....[2]

IGCSE Exam Answer 4

Question	Answer	Marks
11(a)	☐ Holds address of next/current instruction ... ☐ ... to be fetched/processed/executed	2
11(b)	☐ Stores data/instruction that is in use ... ☐ ... from address in MAR	2

Computer Science

Paper 2 - Chapter 7: Algorithm design and problem-solving

7.4 Algorithm design and problem-solving

4 - Understand standard methods of solution.

- linear search
- bubble sort

Paper 2- Learning Objectives

7 Algorithm design and problem-solving continued

Candidates should be able to:

4 Understand standard methods of solution

Notes and guidance

- Limited to:
 - linear search
 - bubble sort
 - totalling
 - counting
 - finding maximum, minimum and average values

7.4 Algorithm design and problem-solving

4 - Understand standard methods of solution.

Linear Search:

<https://www.youtube.com/watch?v=OqOb7hQ1cMw>



7.4 Algorithm design and problem-solving

4 - Understand standard methods of solution.

Linear Search:



Linear search

Starting from the beginning of a data set, each item is checked in turn to see if it is the one being searched for.

A linear search:

- Doesn't require data to be in order.
- Works on any type of storage device.
- Is efficient for small data sets.
- Is very inefficient for large data sets.



Linear search is covered in detail with fully coded examples from page 146 of our book – more information at the end of this video.

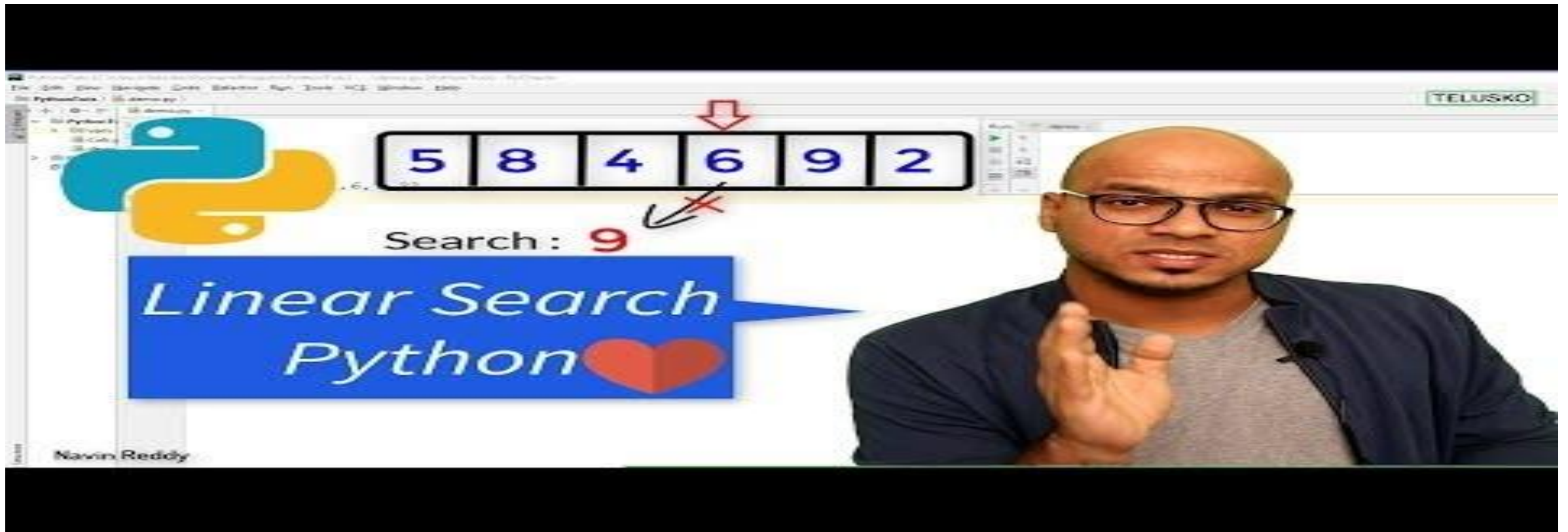
7.4 Algorithm design and problem-solving

4 - Understand standard methods of solution.

Linear Search:

Click the link below the watch the video for a linear search

<https://www.youtube.com/watch?v=UldZOLylez4>



7.4 Algorithm design and problem-solving

4 - Understand standard methods of solution.

Linear Search:

Open Python IDLE and copy the code below for more practical activities

Example 1:

area.py - C:/Users/v90001010/AppData/Local/Programs/Python/Python37-32/area.py (3.7)

File Edit Format Run Options Window Help

```
def linearsearch(arr, x):  
    for i in range(len(arr)):  
        if arr[i] == x:  
            return i  
    return -1  
arr = ['N', 'O', 'U', 'R', 'E', 'D', 'D', 'I', 'N', 'E']  
x = 'I'  
print("element found at index "+str(linearsearch(arr,x)))
```

```
'''  
RESTART: C:/Users/v900010  
element found at index 7  
'''
```

7.4 Algorithm design and problem-solving

4 - Understand standard methods of solution.

Bubble Sort:

<https://www.youtube.com/watch?v=18O0361--1E>



7.4 Algorithm design and problem-solving

4 - Understand standard methods of solution.

Bubble Sort:

<https://www.youtube.com/watch?v=IVBYtXxuZiM>



7.4 Algorithm design and problem-solving

4 - Understand standard methods of solution.

Bubble Sort:

<https://www.youtube.com/watch?v=Vca808JTbI8>



7.4 Algorithm design and problem-solving

4 - Understand standard methods of solution.

Bubble Sort:

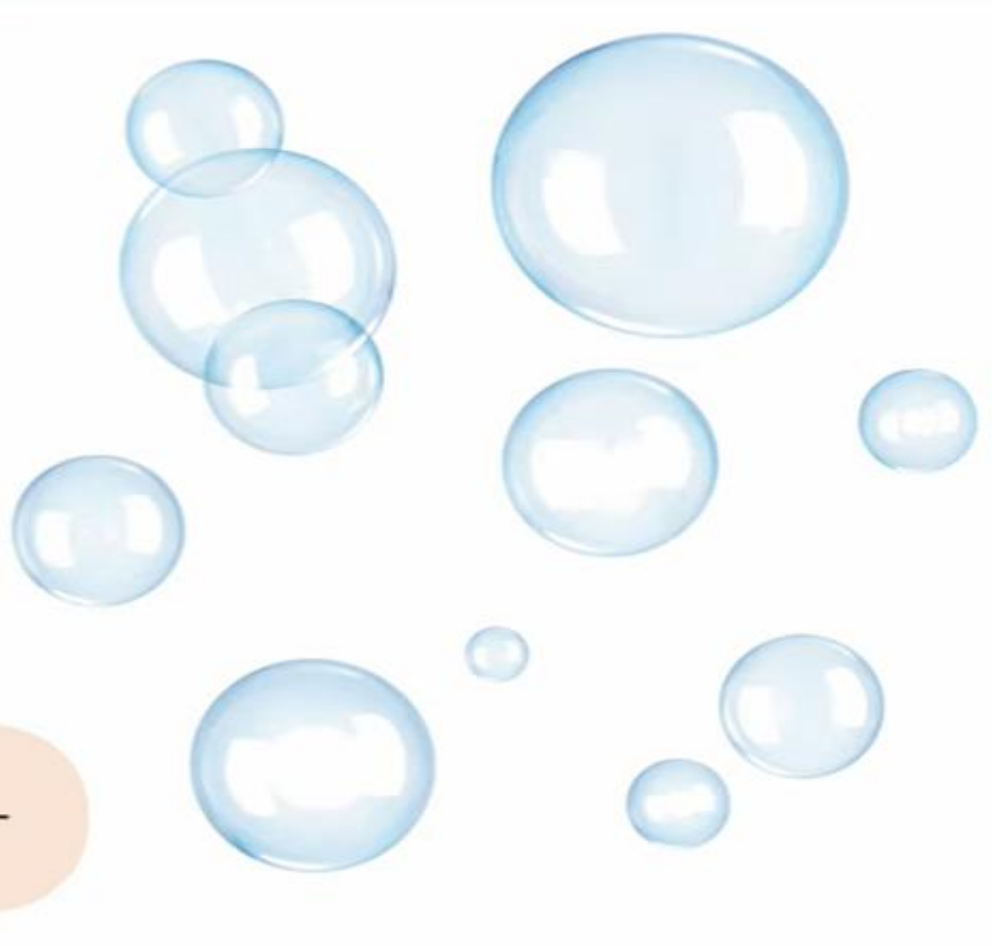


Bubble sort

- Sorts an unordered list of items.
- Compares each item with the next and swaps them if they are out of order – in effect, *bubbling* the largest (or smallest) item up to the end of the list.
- Finishes when no more swaps need to be made.
- The most inefficient of the sorting algorithms but very easy to implement.
- A popular choice for very small data sets.



Bubble sort is covered in detail with fully coded examples from page 154 of our book – more information at the end of this video.



7.4 Algorithm design and problem-solving

4 - Understand standard methods of solution.

Bubble Sort:

How Bubble Sort Works?

We take an unsorted array for our example. Bubble sort takes $O(n^2)$ time so we're keeping it short and precise.



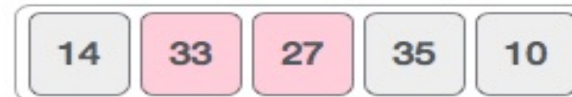
Bubble sort starts with very first two elements, comparing them to check which one is greater.



In this case, value 33 is greater than 14, so it is already in sorted locations. Next, we compare 33 with 27.



We find that 27 is smaller than 33 and these two values must be swapped.



The new array should look like this –



7.4 Algorithm design and problem-solving

4 - Understand standard methods of solution.

Bubble Sort:

How Bubble Sort Works?

Next we compare 33 and 35. We find that both are in already sorted positions.



Then we move to the next two values, 35 and 10.



We know then that 10 is smaller 35. Hence they are not sorted.



We swap these values. We find that we have reached the end of the array. After one iteration, the array should look like this –



To be precise, we are now showing how an array should look like after each iteration. After the second iteration, it should look like this –



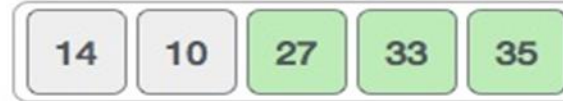
7.4 Algorithm design and problem-solving

4 - Understand standard methods of solution.

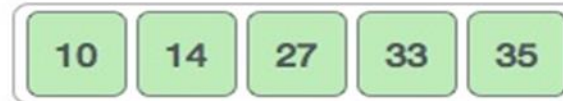
Bubble Sort:

How Bubble Sort Works?

Notice that after each iteration, at least one value moves at the end.



And when there's no swap required, bubble sort learns that an array is completely sorted.



Now we should look into some practical aspects of bubble sort.

Algorithm

We assume **list** is an array of **n** elements. We further assume that **swap** function swaps the values of the given array elements

```
begin BubbleSort(list)

  for all elements of list
    if list[i] > list[i+1]
      swap(list[i], list[i+1])
    end if
  end for

  return list

end BubbleSort
```

7.4 Algorithm design and problem-solving

4 - Understand standard methods of solution.

Bubble Sort:

Open Python IDLE and copy the code below for more practical activities

Example 1:

area.py - C:/Users/v90001010/AppData/Local/Programs/Python/Python37-32/area.py

File Edit Format Run Options Window Help

```
import time, random

l = [5,7,1,4,6,2,3,5,44]

def bubble_sort(l):
    while True:
        swapped = False
        for index in range(len(l)-1):
            if l[index] > l[index+1]:
                l[index], l[index+1] = l[index+1], l[index]
                swapped = True

        if swapped == False:
            return

bubble_sort(l)

print(l)
```

RESTART: C:/Users/v90001010/A:

[1, 2, 3, 4, 5, 5, 6, 7, 44]

...